

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC THĂNG LONG



KHÓA LUẬN TỐT NGHIỆP

XÂY DỰNG CƠ CHẾ TỰ PHỤC HỒI KIỂM THỬ UI WEB DỰA TRÊN MÔ HÌNH TƯƠNG ĐỒNG ĐA CHIỀU VÀ TỐI ƯU TRỌNG SỐ BẰNG MÔ HÌNH HỌC MÁY

KHOA:

CÔNG NGHỆ THÔNG TIN

SINH VIÊN THỰC HIỆN:

TRẦN THỊ NGỌC LAN

GIẢNG VIÊN HƯỚNG DẪN:

ThS. ĐINH THỊ THÚY

HÀ NỘI – 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC THĂNG LONG



KHÓA LUẬN TỐT NGHIỆP

XÂY DỰNG CƠ CHẾ TỰ PHỤC HỒI KIỂM THỬ UI WEB DỰA TRÊN MÔ HÌNH TƯƠNG ĐỒNG ĐA CHIỀU VÀ TỐI ƯU TRỌNG SỐ BẰNG MÔ HÌNH HỌC MÁY

SINH VIÊN THỰC HIỆN:

TRẦN THỊ NGỌC LAN

MÃ SINH VIÊN:

A44984

NGÀNH:

TRÍ TUỆ NHÂN TẠO

GIẢNG VIÊN HƯỚNG DẪN:

ThS. ĐINH THỊ THÚY

HÀ NỘI – 2026

LỜI CẢM ƠN

Để hoàn thành được khóa luận tốt nghiệp này, em đã nhận được sự giúp đỡ từ rất nhiều phía. Em xin chân thành cảm ơn tới tất cả những người đã đồng hành cùng em trong suốt quá trình thực hiện đề tài.

Trước tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Cô ThS. Đinh Thị Thúy- giảng viên hướng dẫn trực tiếp của em. Trong suốt quá trình thực hiện khóa luận, Cô đã tận tình định hướng, chỉ bảo và hỗ trợ em vượt qua những khó khăn về mặt kỹ thuật cũng như phương pháp nghiên cứu. Ngoài ra, cô còn chia sẻ những tài liệu liên quan đến đề tài giúp em hiểu hơn về đề tài em đã chọn. Những đóng góp quý báu của Cô đã giúp em hoàn thiện khóa luận này một cách tốt nhất.

Em cũng xin trân trọng cảm ơn Ban chủ nhiệm Khoa Công nghệ Thông tin, Trường Đại học Thăng Long, cùng toàn thể quý Thầy Cô trong khoa đã trang bị cho em nền tảng kiến thức vững chắc trong suốt bốn năm học qua. Những kiến thức kiểm thử phần mềm, phát triển ứng dụng web và trí tuệ nhân tạo mà em đã được học tập chính là nền móng quan trọng để hoàn thành đề tài này.

Cuối cùng, em xin dành lời tri ân sâu sắc đến gia đình- những người thân yêu luôn bên cạnh, động viên và tạo mọi điều kiện thuận lợi nhất để em có thể toàn tâm, toàn ý học tập và nghiên cứu. Sự yêu thương và ủng hộ của gia đình chính là nguồn động lực lớn lao giúp em vượt qua mọi thử thách.

Mặc dù đã hết sức cố gắng, khóa luận này chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được ý kiến đóng góp từ quý Thầy Cô và các bạn để công trình được hoàn thiện hơn.

Hà Nội, ngày 9 tháng 5 năm 2026

Sinh viên thực hiện

Lan

Trần Thị Ngọc Lan

LỜI CAM ĐOAN

Em tên là: Trần Thị Ngọc Lan

MSV: A44984

Ngành: Trí tuệ nhân tạo

Đề tài Khóa luận tốt nghiệp: “Xây dựng cơ chế tự phục hồi kiểm thử UI web dựa trên mô hình tương đồng đa chiều và tối ưu trọng số bằng mô hình học máy”

Em xin cam đoan rằng khóa luận tốt nghiệp với đề tài “Xây dựng cơ chế tự phục hồi kiểm thử UI web dựa trên mô hình tương đồng đa chiều và tối ưu trọng số bằng mô hình học máy” là công trình nghiên cứu của riêng em, được thực hiện dưới sự hướng dẫn khoa học của ThS.Đinh Thị Thúy.

Em xin cam đoan các nội dung sau:

1. Các số liệu, kết quả thực nghiệm được trình bày trong khóa luận là trung thực và chưa từng được công bố trong bất kỳ công trình nào khác.
2. Những nội dung tham khảo và kế thừa từ các nguồn tài liệu khác đều được trích dẫn đầy đủ và ghi rõ nguồn gốc trong danh mục tài liệu tham khảo.
3. Đề tài không sao chép từ bất kỳ khóa luận, luận văn hay công trình nghiên cứu nào đã có trước đây dưới bất kỳ hình thức nào.

Nếu phát hiện có sự gian lận, em xin hoàn toàn chịu trách nhiệm trước Hội đồng, Nhà trường và pháp luật.

Hà Nội, ngày 9 tháng 5 năm 2026

Sinh viên thực hiện

Lan

Trần Thị Ngọc Lan

TÓM TẮT KHÓA LUẬN

Trong bối cảnh phát triển phần mềm hiện đại, ứng dụng web thay đổi liên tục về giao diện do yêu cầu nghiệp vụ, cập nhật framework và tái cơ cấu thiết kế. Điều này dẫn đến tình trạng các kịch bản kiểm thử tự động dựa trên Selenium thất bại thường xuyên vì locator (ID, XPath, CSS Selector) không còn tìm thấy phần tử trên DOM. Bảo trì bộ test tốn kém và làm chậm vòng lặp CI/CD.

Khóa luận này đề xuất và triển khai một cơ chế tự phục hồi (self-healing) tích hợp trực tiếp vào Selenium WebDriver, hoạt động hoàn toàn tự động mà không cần sửa test thủ công. Hệ thống sử dụng mô hình đánh giá tương đồng phần tử đa chiều gồm 5 chiều: Tương đồng thuộc tính (Attribute Similarity), Tương đồng ngữ nghĩa (Semantic Similarity), Tương đồng cấu trúc DOM (Structure Similarity), Tương đồng trực quan (Visual Similarity) và Tương đồng ngữ cảnh (Context Similarity).

Trọng số của 5 chiều được tối ưu hóa bằng thuật toán Logistic Regression huấn luyện trên dữ liệu healing thực tế. So sánh với phương pháp SAW/AHP truyền thống trên 98 healing events sau khi huấn luyện, Model tối ưu cải thiện score gap lên ~14%, tăng số lần AUTO_FIX từ 76 lên 92 trong khi SUGGEST giảm từ 22 xuống 6, đồng thời đạt Top-1 Success Rate 100%. Kiến trúc gồm 5 module: Exception Interceptor, Snapshot Store, Similarity Engine, Candidate Ranker và Knowledge Base.

Hệ thống được kiểm chứng trên ứng dụng web thương mại điện tử thực nghiệm với 198 healing events qua 10 phiên bản mutation UI. So sánh mô hình 5 chiều với mô hình 3 chiều (bỏ Visual và Context) trên toàn bộ 198 events cho thấy hệ thống 5 chiều đạt accuracy 100% trong khi 3 chiều chỉ đạt 98.48% (3 events heal sai). Kết quả khẳng định tầm quan trọng của thông tin trực quan và ngữ cảnh trong việc nhận diện phần tử.

Hạn chế của hệ thống: hiện tại, hệ thống chỉ được triển khai trên 1 ứng dụng demo đơn lẻ với dữ liệu nhỏ chưa có tính khái quát sang ứng dụng thực tế khác. Các kết quả nghiên cứu chỉ dựa trên dữ liệu thu thập được trên ứng dụng demo này.

DANH MỤC CÁC CHỮ VIẾT TẮT

Viết tắt	Nghĩa đầy đủ
AI	Artificial Intelligence - Trí tuệ nhân tạo
AHP	Analytic Hierarchy Process - Phân tích thứ bậc
API	Application Programming Interface - Giao diện lập trình ứng dụng
DOM	Document Object Model - Mô hình đối tượng tài liệu
JSX	JavaScript XML - Cú pháp mở rộng JavaScript trong React
HTML	HyperText Markup Language - Ngôn ngữ đánh dấu siêu văn bản
SAW	Simple Additive Weighting - Tổng hợp tuyến tính có trọng số
KB	Knowledge Base - Cơ sở tri thức
LR	Logistic Regression - Hồi quy Logistic
ML	Machine Learning - Học máy
SQLite	Lightweight Relational Database (dạng file)
ROC-AUC	Receiver Operating Characteristic - Area Under Curve - Đường cong đặc tính hoạt động – Diện tích dưới đường cong
UI	User Interface - Giao diện người dùng
Xpath	XML Path Language - Ngôn ngữ định vị phần tử XML/HTML
CI/CD	Continuous Integration/Continuous Deployment - Tích hợp và triển khai liên tục

Viết tắt	Nghĩa đầy đủ
CSS	Cascading Style Sheets - Ngôn ngữ định dạng trang web
SPA	Single Page Application - Ứng dụng trang đơn
NLP	Natural Language Processing - Xử lý ngôn ngữ tự nhiên

Mục lục

CHƯƠNG 1. GIỚI THIỆU	1
1.1 Sự cấp thiết của bài toán/đề tài	1
1.2 Mục tiêu nghiên cứu.....	1
1.3 Phạm vi nghiên cứu	2
1.4 Phương pháp tiếp cận	2
1.5 Đóng góp của khóa luận.....	3
1.6 Cấu trúc khóa luận.....	3
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÁC PHƯƠNG PHÁP LIÊN QUAN....	5
2.1 Cơ sở lý thuyết	5
2.1.1 Học máy có giám sát.....	5
2.1.2 Kiểm thử giao diện người dùng	5
2.1.3 Self Healing Test – Khái niệm lịch sử	5
2.1.4 Khái niệm về Selenium WebDriver.....	6
2.1.5 Khái niệm về DOM và locator.....	7
2.2 Các phương pháp tiếp cận.....	8
2.2.1 Tổng quan về SAW và AHP	8
2.2.2 Các phương pháp đo tương đồng văn bản và chuỗi.....	10
2.2.3 Các kiến trúc hệ thống self-healing phổ biến	10
2.2.4 Các hệ thống tương tự	11
2.2.5 Hạn chế của các nghiên cứu trước	12
2.3 Các mô hình AI liên quan.....	12
2.3.1 Logistic Regression.....	12

2.4	Hướng tiếp cận của khóa luận.....	14
CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT		15
3.1	Mô tả quy trình nghiệp vụ	15
3.2	Kiến trúc tổng thể hệ thống	15
3.3	Module 1 – Exception Interceptor	16
3.4	Module 2 – Snapshot Store	17
3.5	Module 3 – Similarity Engine.....	18
3.5.1	Chiều 1 – Attribute Score	18
3.5.2	Chiều 2 – Semantic Score	19
3.5.3	Chiều 3 – Structure Score.....	20
3.5.4	Chiều 4 – Visual Score	20
3.5.5	Chiều 5 – Context Score	21
3.5.6	Công thức tổng hợp SAW.....	22
3.6	Module 4 – Candidate Ranker	29
3.7	Module 5 – Knowledge Base.....	30
3.8	Thiết kế cơ chế tối ưu trọng số bằng Logistic Regression	30
3.8.1	Chuẩn bị dữ liệu	30
3.8.2	Phân chia dữ liệu.....	32
3.8.3	Huấn luyện mô hình.....	32
3.8.4	Chuyển đổi sang trọng số tối ưu.....	32
3.9	Thiết kế ngưỡng quyết định	32
3.9.1	Bộ dữ liệu phân tích.....	32
3.9.2	Phân tích khả năng phân tách	33

3.9.3	<i>Phân tích phân phối điểm số</i>	34
3.9.4	<i>Lựa chọn ngưỡng</i>	34
3.10	Thiết kế cơ sở dữ liệu	35
CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ		37
4.1	Môi trường thực nghiệm	37
4.2	Ứng dụng web thực nghiệm	37
4.3	Bộ dữ liệu thực nghiệm	38
4.4	Các phiên bản mutation UI	38
4.5	Chỉ số đánh giá	39
4.6	Huấn luyện mô hình tối ưu trọng số	39
4.6.1	<i>Dataset sử dụng</i>	39
4.6.2	<i>Xây dựng dữ liệu huấn luyện</i>	40
4.6.3	<i>Lựa chọn mô hình</i>	41
4.6.4	<i>Huấn luyện và tính trọng số tối ưu</i>	41
4.7	So sánh hệ thống sử dụng trọng số tính (SAW/AHP) và hệ thống khi sử dụng trọng số được tối ưu bằng mô hình học máy	43
4.8	So sánh mô hình 5 chiều và mô hình 3 chiều	45
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		48
5.1	Kết luận	48
5.2	Hạn chế của nghiên cứu	48
5.3	Hướng phát triển trong tương lai	49
5.3.1	<i>Cải thiện mô hình tương đồng</i>	49
5.3.2	<i>Cải thiện mô hình học máy</i>	49
5.3.3	<i>Mở rộng phạm vi</i>	50

5.3.4	<i>Tối ưu hiệu năng</i>	50
	TÀI LIỆU THAM KHẢO	51

Danh mục hình ảnh

Hình 3.1 Luồng hệ thống.....	15
Hình 3.2 Phân phối số ứng viên của các event.....	33
Hình 3.3 Phân phối điểm số 1	34
Hình 3.4 Top 15 bộ ngưỡng tối ưu.....	35
Hình 4.1 Trọng số sau 5 lần train	42
Hình 4.2 So sánh Top-1 Accuracy	43
Hình 4.3 So sánh score gap trung bình.....	43
Hình 4.4 Score theo từng step	44
Hình 4.5 Score trước và sau train	44
Hình 4.6 So sánh Auto_fix và suggest	45
Hình 4.7 So sánh Accuracy 5 chiều và 3 chiều	46
Hình 4.8 So sánh những heal event bị heal sai ở mô hình 3 chiều với 5 chiều.....	46

Danh mục bảng

Bảng 3.1 Luồng xử lý chính của hệ thống.....	16
Bảng 3.2 Phân loại các Exception	17
Bảng 3.3 Các trường thông tin trong ElementSnapshot.....	18
Bảng 3.4 Trọng số thuộc tính trong Attribute	19
Bảng 3.5 Trọng số thuộc tính trong Semantic	20
Bảng 3.6 Trọng số thuộc tính trong Structure	20
Bảng 3.7 Tiêu chí đánh giá.....	23
Bảng 3.8 Điểm mức độ phù hợp.....	23
Bảng 3.9 Đánh giá các tiêu chí	24
Bảng 3.10 Hệ số sau khi chuẩn hóa của các chiều	26
Bảng 3.11 Thang điểm Saaty.....	27
Bảng 3.12 So sánh các tiêu chí theo thang điểm Saaty	27
Bảng 3.13 So sánh các tiêu chí sau khi chuẩn hóa	28
Bảng 3.14 Ngưỡng điểm quyết định	29
Bảng 4.1 Môi trường thực nghiệm	37
Bảng 4.2 Bộ dữ liệu thực nghiệm.....	38
Bảng 4.3 Các phiên bản mutation UI	39
Bảng 4.4 Mô tả thuộc tính dữ liệu	40
Bảng 4.5 So sánh lựa chọn model	41
Bảng 4.6 Kết quả train lần đầu	42

CHƯƠNG 1. GIỚI THIỆU

1.1 Sự cấp thiết của bài toán/đề tài

Kiểm thử giao diện người dùng tự động (UI automation testing) đã trở thành một phần không thể thiếu trong quy trình CI/CD hiện đại. Selenium WebDriver là framework được sử dụng rộng rãi nhất, cho phép viết kịch bản kiểm thử tự động mô phỏng hành vi người dùng thực tế trên trình duyệt web.

Tuy nhiên, ứng dụng web trong môi trường phát triển Agile thay đổi liên tục về giao diện do yêu cầu nghiệp vụ, cập nhật framework và tái cơ cấu thiết kế. Điều này dẫn đến tình trạng các kịch bản kiểm thử thất bại thường xuyên vì locator (ID, XPath, CSS Selector) không còn tìm thấy phần tử trên DOM. Vấn đề này sẽ trở nên nghiêm trọng hơn khi dự án có quy mô lớn với hàng trăm test case, hoặc khi team frontend liên tục cập nhật giao diện trong các sprint ngắn.

Khoảng trống nghiên cứu hiện tại: Qua khảo sát tài liệu thì hiện tại các nghiên cứu tiếp cận bài toán self-healing theo nhiều chiều khác nhau. WATER [3], WATERFALL [4] và Healenium [13] chủ yếu dựa trên thông tin thuộc tính, cấu trúc DOM để phục hồi locator. Yandrapally và cộng sự [14] đề xuất khai thác Exception các contextual clues nhằm tăng khả năng định vị phần tử khi giao diện thay đổi, trong khi Gong và cộng sự [12] sử dụng đặc trưng thị giác của giao diện để thực hiện self-healing. Tuy nhiên, các phương pháp này thường tập trung vào một, hai hoặc ba chiều chưa thấy có hệ thống nào kết hợp đồng thời 5 chiều tương đồng với tối ưu trọng số bằng học máy trong một hệ thống tích hợp liền mạch. Bên cạnh đó, nghiên cứu hiện tại mới thực hiện trên một ứng dụng demo đơn lẻ với dữ liệu nhỏ, chưa có tính khái quát sang ứng dụng thực tế phức tạp hơn.

Khóa luận này đề xuất một cơ chế tự phục hồi (self-healing) toàn diện, tích hợp trực tiếp vào Selenium WebDriver, sử dụng mô hình đo tương đồng 5 chiều kết hợp với Logistic Regression để tự động sửa locator bị hỏng mà không cần can thiệp thủ công.

1.2 Mục tiêu nghiên cứu

Mục tiêu tổng quát của khóa luận là xây dựng cơ chế tự phục hồi cho bộ kiểm thử UI web tự động, có khả năng phát hiện và sửa chữa các locator bị hỏng do thay đổi giao diện mà không cần sự can thiệp thủ công.

Để đạt được mục tiêu đó, khóa luận hướng đến bốn mục tiêu cụ thể sau:

- Xây dựng và kiểm chứng mô hình tương đồng 5 chiều (Attribute, Semantic, Structure, Visual, Context) có đủ khả năng nhận diện chính xác phần tử thay thế khi locator gốc thất bại do thay đổi UI. Đồng thời, đánh giá mức độ đóng góp của hai chiều Visual và Context thông qua so sánh thực nghiệm giữa mô hình 5 chiều

và mô hình 3 chiều. Hiệu quả của mô hình được đánh giá bằng chỉ số Top-1 Accuracy trên tập healing event thực nghiệm và số lượng healing event được phục hồi chính xác.

- So sánh và đánh giá hai phương pháp tổng hợp trọng số - trọng số tĩnh được tính bằng công thức SAW/AHP và trọng số tối ưu học bởi Logistic Regression - để xác định phương pháp tối ưu trọng số bằng model học máy cho khả năng phân biệt candidate đúng/sai tốt hơn. Việc đánh giá được thực hiện thông qua các chỉ số như Top-1 Accuracy, Score Gap, Margin Distribution và tỷ lệ Auto-fix/Suggest
- Triển khai và kiểm chứng hệ thống self-healing hoàn chỉnh tích hợp với Selenium WebDriver, đánh giá hiệu quả thực tế trên ứng dụng web thực nghiệm với 198 healing events qua 10 phiên bản mutation UI.

1.3 Phạm vi nghiên cứu

Về phạm vi chức năng, hệ thống tập trung vào kiểm thử giao diện ứng dụng web SPA (Single Page Application) xây dựng bằng React.js. Loại mutation được xử lý bao gồm: đổi tên thuộc tính định danh, thay đổi loại thuộc tính, xóa thuộc tính, thay đổi văn bản hiển thị, thêm wrapper element và tổ hợp đa chiều.

- Về nội dung: Hệ thống tập trung xử lý các loại mutation phổ biến (đổi tên/loại/xóa thuộc tính, thay đổi văn bản, thêm wrapper element). Các thay đổi layout hoàn toàn hoặc thêm màn hình mới nằm ngoài phạm vi. Semantic similarity sử dụng SequenceMatcher, chưa áp dụng NLP/embedding.
- Về kỹ thuật: Hệ thống tích hợp với Selenium WebDriver qua kiến trúc Exception Interceptor, hỗ trợ ứng dụng SPA React.js chạy trên Chrome. Visual score yêu cầu phần tử phải hiển thị và có bounding box hợp lệ; phần tử ẩn, off-screen hoặc lazy-loaded có thể cho điểm không chính xác.
- Về phạm vi dữ liệu: Thực nghiệm được tiến hành trên một ứng dụng SPA React.js với 198 healing events thu thập từ 10 phiên bản mutation UI có kiểm soát. Tính khái quát hóa sang ứng dụng thực tế phức tạp hơn chưa được kiểm chứng.
- Về giới hạn: Nghiên cứu được thực hiện trên một ứng dụng demo đơn lẻ trong môi trường kiểm soát, do đó kết quả chưa phản ánh đầy đủ độ phức tạp của hệ thống thực tế. Trọng số học được từ Logistic Regression và bộ ngưỡng quyết định (AUTO_FIX=0.70, SUGGEST=0.50) có thể không tối ưu khi áp dụng sang ứng dụng có đặc trưng phân phối điểm khác biệt. Multicollinearity giữa các sub-score cũng khiến trọng số ML không hoàn toàn ổn định khi dữ liệu thay đổi.

1.4 Phương pháp tiếp cận

Khóa luận kết hợp nhiều phương pháp nghiên cứu:

- Nghiên cứu tài liệu: Phân tích các công trình liên quan về self-healing test, kỹ thuật sửa locator và nhận diện phần tử dựa trên tương đồng. Trong các công trình và nghiên cứu trước đó thì chưa có giải pháp nào kết hợp đồng thời 5 chiều tương đồng với tối ưu trọng số bằng học máy.
- Thiết kế và cài đặt hệ thống: Xây dựng 5 module tích hợp liên mạch với Selenium WebDriver (Exception Interceptor, Snapshot Store, Similarity Engine, Candidate Ranker, Knowledge Base), sử dụng Python, SQLite, scikit-learn.
- Thực nghiệm có kiểm soát: Sử dụng mutation engine tự động tạo 10 phiên bản UI biến thể từ ứng dụng baseline, thu thập 198 healing events thực tế.
- Nghiên cứu so sánh định lượng: So sánh có kiểm soát (1) SAW/AHP vs. Model tối ưu trên 98 events post-training; (2) 5 chiều vs. 3 chiều trên toàn bộ 198 events, sử dụng các chỉ số Top-1 Success Rate, Score Gap, ROC-AUC, Recall.

1.5 Đóng góp của khóa luận

Khóa luận có các đóng góp chính sau:

- Đề xuất mô hình tương đồng 5 chiều toàn diện với công thức tính điểm chi tiết cho từng chiều, kết hợp đo tương đồng chuỗi (SequenceMatcher, Jaccard), tương đồng cấu trúc DOM và tương đồng trực quan (bounding box). Mô hình đạt Top-1 Success Rate 100% trên bộ dữ liệu thực nghiệm tự thu thập (198 healing events).
- Tích hợp Logistic Regression học trọng số 5 chiều từ dữ liệu healing thực tế theo phương pháp pairwise, vượt trội hơn SAW/AHP tĩnh: score gap tăng 14%, AUTO_FIX tăng từ 76 lên 92 trên 98 events post-training.
- Xây dựng bộ dữ liệu healing thực tế gồm 198 events và 750 candidate records thu thập từ 10 phiên bản mutation UI có kiểm soát.
- Cung cấp bằng chứng thực nghiệm (trên dataset thu được từ 1 demo đơn lẻ): mô hình 5 chiều đạt accuracy 100% trong khi 3 chiều đạt 98.48% (3 events sai), cho thấy tầm quan trọng của Visual và Context score, đặc biệt trong các mutation không thay đổi vị trí trực quan phần tử.

1.6 Cấu trúc khóa luận

Khóa luận được tổ chức thành 5 chương chính như sau:

- Chương 1 — Giới thiệu: Trình bày sự cấp thiết của bài toán, mục tiêu, phạm vi, phương pháp tiếp cận và các đóng góp chính.
- Chương 2 — Cơ sở lý thuyết và các phương pháp liên quan: Trình bày nền tảng lý thuyết AI/ML, các phương pháp tiếp cận hiện có trong lĩnh vực self-healing test, các mô hình/công cụ liên quan và hướng tiếp cận của khóa luận.

- Chương 3 — Phương pháp đề xuất: Mô tả kiến trúc hệ thống 5 module, bộ dữ liệu, các bước tiền xử lý, mô hình tương đồng 5 chiều và chiến lược học trọng số bằng Logistic Regression.
- Chương 4 — Thực nghiệm và đánh giá: Trình bày môi trường thực nghiệm, kết quả huấn luyện mô hình, so sánh trọng số tĩnh được tính bằng công thức SAW/AHP với Model tối ưu và so sánh 5 chiều với 3 chiều.
- Chương 5 — Kết luận và hướng phát triển: Tóm tắt kết quả, hạn chế và đề xuất các hướng nghiên cứu tiếp theo.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÁC PHƯƠNG PHÁP LIÊN QUAN

2.1 Cơ sở lý thuyết

2.1.1 Học máy có giám sát

Học máy có giám sát là nhánh của trí tuệ nhân tạo trong đó mô hình được huấn luyện trên tập dữ liệu có nhãn để học ánh xạ từ đầu vào sang đầu ra. Trong đề tài này, bài toán học có giám sát được đặt ra ở cấp độ tối ưu trọng số: hệ thống học cách gán tầm quan trọng cho từng chiều tương đồng dựa trên lịch sử healing thực tế (198 healing events, 750 mẫu candidate), trong đó nhãn is_correct cho biết candidate nào là phần tử đúng cần phục hồi.

2.1.2 Kiểm thử giao diện người dùng

Kiểm thử giao diện người dùng tự động (automated UI testing) là phương pháp sử dụng phần mềm để tự động hóa việc thực thi và kiểm tra hành vi giao diện của ứng dụng web. Thay vì người kiểm thử tương tác thủ công với ứng dụng, các kịch bản kiểm thử được viết thành code và thực thi bởi framework kiểm thử.

Selenium WebDriver là framework kiểm thử UI phổ biến nhất hiện nay, cho phép điều khiển trình duyệt web thông qua API lập trình. Cơ chế hoạt động của Selenium dựa trên khái niệm 'locator' - một chuỗi định vị xác định vị trí của một phần tử trong DOM. Các loại locator phổ biến bao gồm: ID, Name, XPath, CSS Selector, Class Name, Link Text và Tag Name.

Vấn đề cốt lõi của kiểm thử UI tự động là 'locator fragility' (tính dễ gãy của locator). Khi giao diện thay đổi - dù chỉ là đổi tên một thuộc tính HTML - locator trong test case không còn tìm thấy phần tử, gây ra NoSuchElementException và làm test fail.

2.1.3 Self Healing Test – Khái niệm lịch sử

Self-healing test là cơ chế mà hệ thống kiểm thử tự động phát hiện khi một locator thất bại và tự tìm phần tử thay thế trong DOM hiện tại, thay vì dừng lại và báo lỗi cho kỹ sư.

Khái niệm này lần đầu được đề xuất trong các nghiên cứu về 'robust test automation' vào đầu thập niên 2010. Các công cụ thương mại như Healenium, TestIM, mabl và Applitools đã đưa ra các triển khai thực tế. Tuy nhiên, phần lớn các công cụ này sử dụng cách tiếp cận hộp đen (black-box) dựa trên machine learning thuần túy mà không giải thích được lý do chọn phần tử.

Tiếp cận của khóa luận này là hybrid approach: kết hợp kỹ thuật đo tương đồng có giải thích được (explainable similarity scoring) với machine learning để tối ưu trọng số.

Điều này giúp hệ thống vừa có độ chính xác cao, vừa cho phép kỹ sư hiểu và tin tưởng vào quyết định của hệ thống.

2.1.4 Khái niệm về Selenium WebDriver

WebDriver là một giao diện lập trình ứng dụng (API) cho phép thực hiện thao tác trên các trình duyệt web như mở trang web, nhập dữ liệu, nhấn nút, kéo thả, chụp ảnh màn hình... WebDriver được ra đời vào năm 2006 bởi Simon Stewart, một kỹ sư phần mềm. Năm 2008, WebDriver được kết hợp với Selenium RC để tạo ra Selenium 2.0 về sau phát triển tới Selenium 4.x, là phiên bản dùng trong khóa luận [11].

Selenium WebDriver là một thành phần của Selenium, bao gồm cả Selenium IDE, Selenium Grid và Selenium RC. Selenium WebDriver là phiên bản nâng cấp của Selenium RC, với khả năng tương tác trực tiếp với trình duyệt và thực hiện các thao tác như click, nhập liệu, kéo thả, chụp ảnh màn hình,... Selenium WebDriver hỗ trợ nhiều ngôn ngữ lập trình và nhiều trình duyệt khác nhau rất thân thiện và dễ sử dụng.

2.1.4.1 Tại sao nên sử dụng selenium WebDriver?

- WebDriver hoàn toàn miễn phí và dễ sử dụng.
- WebDriver hỗ trợ nhiều ngôn ngữ lập trình khác nhau để có thể viết kịch bản kiểm thử theo ý muốn. Hoặc cũng có thể áp dụng các cấu trúc điều khiển như if else hay vòng lặp để làm cho kịch bản chính xác hơn.
- WebDriver là công cụ nhanh nhất trong bộ Selenium, vì nó giao tiếp trực tiếp với trình duyệt mà không cần qua bất kỳ lớp trung gian nào.

2.1.4.2 Một số khái niệm cần biết trong Selenium WebDriver trong khóa luận

Element là đối tượng đại diện cho một thành phần của giao diện người dùng trên trang web, ví dụ như một nút, một liên kết, một ô nhập liệu,... có thể tìm kiếm các Element trên trang web bằng cách sử dụng các phương thức `find_element_by_()` của đối tượng WebDriver, hoặc bằng cách sử dụng các phương thức `find_elements_by_()` để tìm kiếm nhiều Element cùng một lúc. Để có thể tương tác với các Element bằng cách sử dụng các phương thức như `click()`, `send_keys()`, `clear()`,...

Locator là một cách để xác định vị trí của một Element trên trang web, bằng cách sử dụng một thuộc tính hoặc một biểu thức của Element đó. Có nhiều loại locator khác nhau, ví dụ như ID, name, class name, tag name, CSS selector, XPath,... để có thể sử dụng Locator để truyền vào các phương thức `find_element_by_()` hoặc `find_elements_by_()` để tìm kiếm các Element mong muốn.

2.1.5 Khái niệm về DOM và locator

2.1.5.1 DOM

DOM (viết tắt của Document Object Model - Mô hình Đối tượng Tài liệu) là một giao diện lập trình ứng dụng (API) chuẩn hóa bởi W3C, dùng để truy xuất và thao tác các tài liệu dạng HTML hoặc XML. Khi trình duyệt tải trang web, nó chuyển đổi mã HTML thành một cấu trúc cây logic (cây DOM), trong đó mỗi phần tử (thẻ, văn bản, thuộc tính) là một "nút" (node).

Các thành phần chính

- Document: Nút gốc (root node) đại diện cho toàn bộ tài liệu
- HTML.Element: Các nút đại diện cho các thẻ HTML (ví dụ: <div>, <p>, <input>).
- Attribute: Thuộc tính của thẻ (ví dụ: class, id, name).
- Text: Nội dung văn bản bên trong các thẻ HTML.
- Vai trò
- Thao tác nội dung: Cho phép JavaScript thay đổi văn bản, hình ảnh, hoặc cấu trúc HTML.
- Tương tác người dùng: Phản hồi lại các sự kiện (click, di chuột, nhập liệu).
- Dynamic Update: Cập nhật giao diện mà không cần tải lại trang.

2.1.5.2 Locator

Locator (bộ định vị) là một kỹ thuật hoặc phương pháp dùng để xác định vị trí của một phần tử (element) cụ thể trên trang web (như button, ô nhập liệu, checkbox) dựa trên cấu trúc DOM.

Vai trò của Locator:

- Automation Testing: Dùng để tìm kiếm phần tử để tương tác (click, nhập liệu) trong các công cụ như Selenium, Appium, Cypress.
- Web Scraping: Dùng để cào dữ liệu từ website.

Các loại Locator phổ biến:

- ID: Cách nhanh và chính xác nhất. ID nên là duy nhất trên trang.
- Name: Thường dùng cho các thẻ input, button.
- Class Name: Sử dụng tên class của CSS, có thể trùng nhau.

- Tag Name: Sử dụng thẻ HTML (ví dụ: <a>, <h1>).
- CSS Selector: Cú pháp mạnh mẽ, linh hoạt, tốc độ nhanh, thường được khuyến khích sử dụng.
- XPath: Rất mạnh, có thể tìm kiếm theo cấu trúc cây (con, cha, anh em) hoặc nội dung văn bản.
- Link Text / Partial Link Text: Dùng đặc thù cho các thẻ liên kết <a>

Mối quan hệ giữa DOM và Locator

- DOM là bản đồ, Locator là địa chỉ: DOM cung cấp cấu trúc cây cho phép trình duyệt hiểu trang web.
- Locator giúp lập trình viên/tester tìm đường đến một node (thành phần) cụ thể trong cây đó.
- Tương tác: Dùng Locator (như XPath hoặc CSS) để tìm một node trong DOM, sau đó sử dụng JavaScript hoặc công cụ Automation để thay đổi/thao tác node đó.

2.2 Các phương pháp tiếp cận

2.2.1 Tổng quan về SAW và AHP

2.2.1.1 SAW

SAW, còn được gọi là phương pháp trọng số cộng đơn giản, là một trong những kỹ thuật MCDM đơn giản và được sử dụng rộng rãi nhất [15].

Nguyên lý

SAW dựa trên tổng trọng số có trọng số của các thuộc tính đánh giá cho từng phương án. Nó giả định sự phụ thuộc tuyến tính giữa các thuộc tính, nghĩa là tổng điểm của một phương án là tổng của các giá trị thuộc tính đã chuẩn hóa nhân với trọng số tương ứng của chúng.

Quy trình

1. Xác định tiêu chí như 1 người đánh giá về những ưu điểm của 1 biến số đã cho.
2. Xác định tỉ lệ phù hợp cho mỗi phương án dựa trên các tiêu chí đã cho.
3. Lập ma trận quyết định dựa trên các tiêu chí.
4. Tiến hành chuẩn hóa dựa trên lợi ích và chi phí của từng tiêu chí.
5. Xác định giá trị cuối cùng dựa trên trọng số đã xác định.

Ưu và nhược điểm

- ✓ Ưu điểm: Dễ hiểu, dễ tính toán và nhanh chóng.
- ✗ Nhược điểm: Phụ thuộc vào việc xác định trọng số chính xác và giả định sự tuyến tính (không phản ánh được các mối quan hệ phức tạp giữa các tiêu chí).

Công thức

$$\text{Score} = \sum (w_i \times s_i) \text{ với } \sum w_i = 1$$

Trong đó :

- + w_i là trọng số của tiêu chí i .
- + s_i là điểm của tiêu chí i trong khoảng $[0,1]$.

2.2.1.2 AHP

AHP (Analytic Hierarchy Process) của Saaty [7] là phương pháp xác định trọng số w_i thông qua ma trận so sánh cặp đôi (pairwise comparison matrix). Người chuyên gia đánh giá mức độ quan trọng tương đối của từng cặp tiêu chí theo thang 1-9. Trọng số được tính từ eigenvector chính của ma trận so sánh.

Nguyên lý

AHP chia nhỏ vấn đề ra quyết định phức tạp thành một cấu trúc thứ bậc bao gồm: mục tiêu, tiêu chí, tiêu chí con và các phương án. Sau đó, nó sử dụng so sánh cặp (pairwise comparison) để đánh giá tầm quan trọng tương đối của các yếu tố.

Quy trình thực hiện

1. Xây dựng mô hình thứ bậc.
2. Thực hiện so sánh cặp giữa các yếu tố (sử dụng thang đo từ 1-9).
3. Tính toán trọng số (vector riêng) và kiểm tra tính nhất quán của các so sánh.
4. Tổng hợp kết quả để đưa ra quyết định cuối cùng.

Ưu và nhược điểm

- ✓ Ưu điểm: Có khả năng xử lý các tiêu chí định tính và định lượng, đánh giá tính nhất quán của dữ liệu đầu vào.
- ✗ Nhược điểm: Phức tạp hơn trong tính toán khi số lượng tiêu chí quá lớn, có thể gây khó khăn khi so sánh cặp.

Trong khóa luận này, AHP được dùng để xác định bộ trọng số khởi điểm (bộ SAW/AHP), sau đó so sánh với bộ trọng số tối ưu của ML để đánh giá kết quả.

2.2.2 Các phương pháp đo tương đồng văn bản và chuỗi

2.2.2.1 SequenceMatcher (DiffLib)

Python cung cấp module `difflib.SequenceMatcher` cho phép tính tỷ lệ tương đồng giữa hai chuỗi dựa trên thuật toán tìm chuỗi con chung dài nhất (Longest Common Subsequence). Đây là một trong những kỹ thuật đo tương đồng chuỗi cơ bản được sử dụng rộng rãi trong xử lý văn bản [8]. Công thức tính:

$$\text{ratio} = 2.0 * M / T$$

Trong đó:

- + M là số ký tự khớp.
- + T là tổng số ký tự.

Đây là phương pháp được sử dụng trong khóa luận cho tất cả các so sánh chuỗi scalar (id, text, href, ...).

2.2.2.2 Jaccard Similarity cho tập hợp

Để so sánh danh sách classes (CSS classes), khóa luận sử dụng Jaccard Similarity:

$$J(A,B) = |A \cap B| / |A \cup B|.$$

Công thức này không phụ thuộc vào thứ tự và xử lý tốt trường hợp một trong hai tập rỗng. Độ đo này thuộc nhóm các phương pháp tương đồng tập hợp được sử dụng phổ biến trong khai phá văn bản [8].

2.2.2.3 Phát hiện giá trị động (Dynamic Value Detection)

Một thách thức quan trọng là nhiều thuộc tính HTML (đặc biệt id và data-testid) được sinh ra tự động bởi framework CSS (ví dụ: `css-a1b2c3`, `sc-xyz`) hoặc chứa timestamp/hash dài. Sử dụng các giá trị này để so sánh sẽ cho điểm thấp giả tạo. Hệ thống sử dụng regex patterns để phát hiện và giảm trọng số đóng góp của các giá trị động này.

2.2.3 Các kiến trúc hệ thống self-healing phổ biến

Các hệ thống self-healing hiện có được triển khai theo hai hướng kiến trúc chính:

- Kiến trúc Proxy (Healenium): Đứng giữa test script và WebDriver, chặn request tìm element và tự xử lý. Ưu điểm: trong suốt với test script. Nhược điểm: thêm latency, khó debug, phụ thuộc backend riêng.
- Kiến trúc Exception Interceptor (hướng tiếp cận của khóa luận): Bắt exception `NoSuchElementException` ngay trong driver wrapper, kích hoạt healing pipeline

nội bộ. Ưu điểm: không cần server riêng, tích hợp nhẹ, dễ kiểm soát luồng. Đây là kiến trúc được chọn vì phù hợp hơn với môi trường CI/CD và dễ mở rộng.

2.2.4 Các hệ thống tương tự

- ROBULA+ [1]: Thuật toán tạo XPath bền vững bằng cách generalize dần từ XPath tuyệt đối – bắt đầu từ một XPath rất cụ thể rồi loại bỏ dần các ràng buộc không cần thiết cho đến khi tìm được biểu thức vừa khớp phần tử đúng vừa ổn định trước các thay đổi DOM nhỏ. Hướng tiếp cận này tập trung vào việc tạo locator bền vững từ đầu hơn là phục hồi khi đã hỏng. ROBULA+ không sử dụng cơ chế học từ dữ liệu thực và không xét đến thông tin ngữ nghĩa hay trực quan của phần tử.
- WATER [3] / WATERFALL [4]: Là hai công trình tiên phong trong lĩnh vực tự động sửa test. WATER so sánh thuộc tính HTML và cấu trúc DOM để tìm phần tử thay thế khi locator thất bại. WATERFALL kế thừa ý tưởng này và bổ sung cơ chế incremental — ưu tiên tìm kiếm ở các vùng DOM gần vị trí cũ trước để giảm thời gian phục hồi. Cả hai hệ thống đều giới hạn ở ba chiều tương đồng là thuộc tính, ngữ nghĩa và cấu trúc DOM, không khai thác thông tin trực quan (vị trí, kích thước phần tử trên màn hình) và ngữ cảnh lân cận. Quan trọng hơn, trọng số giữa các chiều được thiết lập tĩnh theo kinh nghiệm.
- PESTO [2]: Tiếp cận bài toán theo hướng hoàn toàn khác — sử dụng hình ảnh màn hình (screenshot) để nhận diện phần tử thay thế thông qua so sánh vùng hình ảnh xung quanh vị trí cũ. Ưu điểm của PESTO là không phụ thuộc vào cấu trúc DOM, hoạt động tốt ngay cả khi toàn bộ thuộc tính HTML bị thay đổi. Tuy nhiên chi phí tính toán cao, dễ bị ảnh hưởng bởi thay đổi theme giao diện, màu sắc hoặc độ phân giải màn hình — những yếu tố không liên quan đến việc phần tử có còn tồn tại hay không.
- Healenium [13]: Là thư viện mã nguồn mở được triển khai rộng rãi nhất trong thực tế hiện nay. Healenium xây dựng cây DOM snapshot trước mỗi lần chạy test, và khi locator thất bại, hệ thống tính điểm tương đồng dựa trên thuật toán tree similarity giữa snapshot cũ và DOM hiện tại. Healenium cung cấp giao diện web để người dùng xem xét và xác nhận kết quả healing, đồng thời hỗ trợ nhiều ngôn ngữ test khác nhau. Điểm hạn chế là Healenium chỉ dựa vào cấu trúc cây DOM và thuộc tính HTML, không khai thác thông tin ngữ nghĩa, trực quan hay ngữ cảnh. Toàn bộ quá trình dựa trên luật cố định mà không có cơ chế tự học trọng số từ lịch sử healing. Không tối ưu trọng số bằng ML.
- Nguyen và cộng sự [6]: Đề xuất tiến hóa tự động locator bằng kỹ thuật tìm kiếm đơn giản (simple search techniques), cho thấy ngay cả chiến lược tìm kiếm nhẹ cũng

cải thiện đáng kể độ bền locator khi giao diện thay đổi. Nghiên cứu này cung cấp bằng chứng quan trọng rằng việc đa dạng hóa chiến lược locator có thể giảm tần suất thất bại test, tuy nhiên không có cơ chế học trọng số đa chiều và không tích hợp thông tin trực quan hay ngữ cảnh.

2.2.5 Hạn chế của các nghiên cứu trước

Qua khảo sát các công trình liên quan thì có nhận thấy ba hạn chế chính tồn tại trong các nghiên cứu hiện có đó là:

Thứ nhất, không có hệ thống nào kết hợp đồng thời cả năm chiều tương đồng. Mỗi nghiên cứu chỉ khai thác một đến 3 chiều, dẫn đến khả năng nhận diện sai trong các trường hợp thay đổi phức tạp.

Thứ hai, trọng số giữa các chiều tương đồng đều được thiết lập tĩnh theo kinh nghiệm hoặc công thức cố định. Không có cơ chế học từ lịch sử healing thực tế, đồng nghĩa với việc hệ thống không tự cải thiện theo thời gian và không thích nghi với đặc trưng riêng của từng ứng dụng.

Thứ ba, thiếu cơ chế phân loại mức độ tự tin trong quyết định healing. Hầu hết các hệ thống chỉ đưa ra một kết quả duy nhất fix hoặc báo lỗi mà không có vùng đệm để người dùng có thể xem xét các trường hợp không chắc chắn.

Nghiên cứu này giải quyết đồng thời ba vấn đề trên thông qua: mô hình tương đồng năm chiều toàn diện, cơ chế học trọng số online bằng Logistic Regression theo phương pháp pairwise, và hệ thống ngưỡng quyết định ba mức (AUTO_FIX/ SUGGEST/ FAILED).

2.3 Các mô hình AI liên quan

2.3.1 Logistic Regression

Logistic Regression (LR) là mô hình học máy tuyến tính cho phân loại nhị phân. Xác suất mẫu thuộc lớp dương tính được tính:

$$P(y=1|x) = \sigma(w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n)$$

Hàm sigmoid $\sigma(z) = 1 / (1 + e^{-z})$ đảm bảo đầu ra trong khoảng (0,1). Ứng dụng trong khóa luận này, $x_1...x_5$ là 5 sub-scores (attr, sem, struct, visual, ctx) và nhãn y là is_correct (1 = candidate đúng, 0 = sai).

Hệ số $w_1...w_5$ học được từ dữ liệu, sau khi chuẩn hóa thành tổng bằng 1, trực tiếp trở thành trọng số w cho công thức SAW. Đây là cách chuyển đổi tự nhiên từ ML về dạng có thể giải thích được.

Tham số `class_weight='balanced'` được sử dụng vì dữ liệu mất cân bằng: mỗi healing event chỉ có 1 candidate đúng nhưng có thể có 3-10 candidate sai.

- ✓ Ưu điểm: Dễ triển khai, tốc độ tính toán nhanh và cung cấp xác suất cụ thể cho từng dự đoán thay vì chỉ đưa ra nhãn.
- ✗ Hạn chế: Chỉ hoạt động tốt khi mối quan hệ giữa biến độc lập và biến phụ thuộc là gần như tuyến tính; khó xử lý các bài toán có dữ liệu phức tạp, phi tuyến tính.

Hệ thống dùng Logistic Regression để học trọng số tối ưu từ dữ liệu healing thực tế.

Hệ số hồi quy w_i của Logistic Regression phản ánh trực tiếp mức độ đóng góp của feature i . Sau khi lấy $|w_i|$ và chuẩn hóa về tổng bằng 1, kết quả chính là trọng số được tối ưu.

2.3.1.1 Các chỉ số đánh giá

2.3.1.1.1 Top-1 Accuracy

Ý nghĩa: Đo tỷ lệ event mà candidate có điểm `total_score` cao nhất chính là candidate đúng (`is_correct = 1`).

Công thức

$$Top - 1 Accuracy = \frac{\text{Số event chọn đúng candidate top 1}}{\text{Tổng số event}}$$

Diễn giải

- 100% = mọi event đều xếp element đúng ở vị trí số 1.
- Đây là chỉ số quan trọng nhất vì mục tiêu là chọn đúng locator để heal.

2.3.1.2 Correct vs Wrong Score Gap (Khoảng cách giữa ứng viên đúng và sai)

Ý nghĩa: Khoảng cách giữa candidate đứng thứ nhất ứng viên đúng và thứ hai ứng viên sai.

Công thức

$$Gap = Avg(\text{Scorecorrect}) - Avg(\text{Scorewrong})$$

Gap càng lớn:

- Candidate đúng càng được đẩy lên cao
- Candidate sai càng bị kéo xuống thấp
- Model càng dễ ra quyết định

Đây là chỉ số đánh giá chất lượng ranking rất tốt.

2.3.1.3 Healing Success Rate

Công thức

$$SuccessRate = \frac{\text{Số event heal thành công}}{\text{Tổng event}}$$

Ý nghĩa

- Đánh giá hiệu quả thực tế của hệ thống.

Khác với Accuracy:

- Accuracy đánh giá candidate ranking.
- Success Rate đánh giá kết quả heal cuối cùng.

2.4 Hướng tiếp cận của khóa luận

Từ việc tổng hợp các nghiên cứu liên quan, khóa luận xác định khoảng trống: chưa có hệ thống nào kết hợp đồng thời 5 chiều tương đồng (trong đó có visual và context) với tối ưu trọng số bằng học máy và Knowledge Base cache.

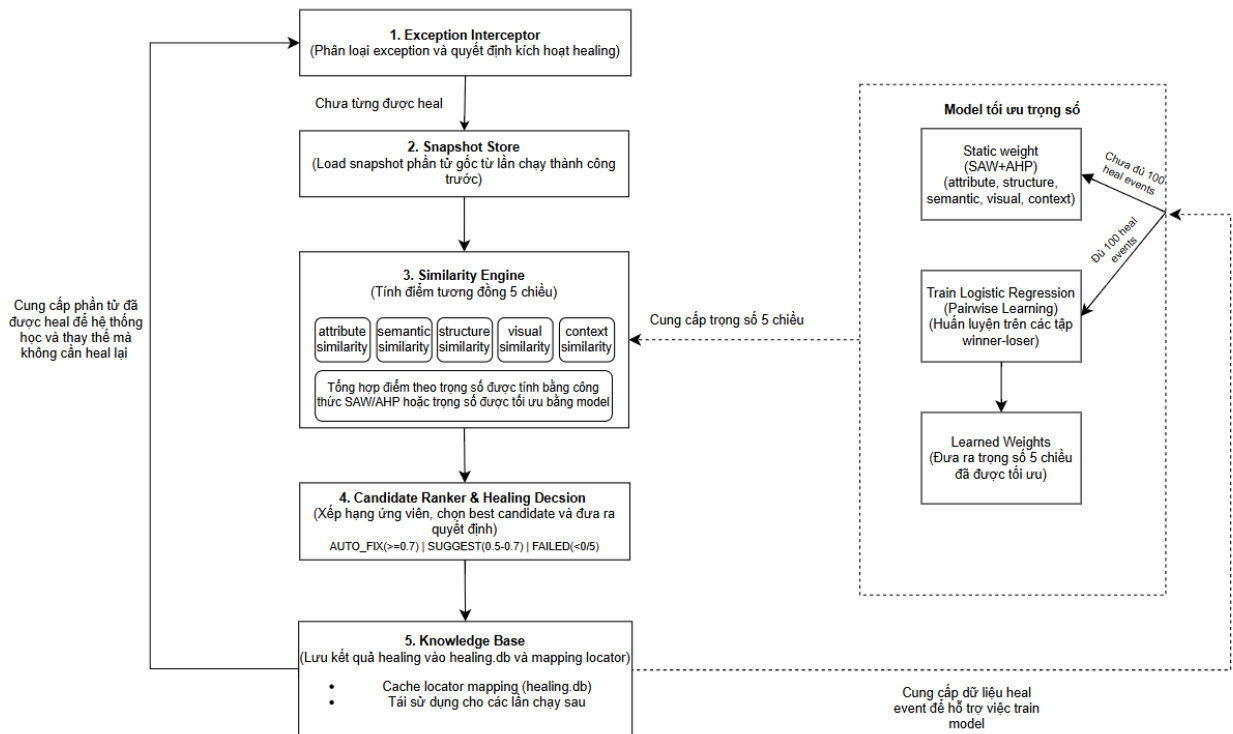
Hướng tiếp cận được chọn: Kiến trúc Exception Interceptor tích hợp vào Selenium driver, với Similarity Engine 5 chiều dùng trọng số tĩnh SAW làm baseline và tự động chuyển sang trọng số Logistic Regression sau khi đủ 100 healing events tích lũy. Phương pháp pairwise training được chọn thay vì phân lớp trực tiếp để đảm bảo model học "candidate nào tốt hơn trong cùng event" thay vì "candidate có đúng hay không theo nghĩa tuyệt đối".

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT

3.1 Mô tả quy trình nghiệp vụ

Quy trình kiểm thử giao diện tự động được phối hợp giữa ba vai trò: đội phát triển giao diện (frontend), hệ thống tích hợp liên tục (CI) và kỹ sư kiểm thử (QA). Chu trình diễn ra như sau: (1) QA định nghĩa kịch bản kiểm thử, mỗi bước trở tới phần tử giao diện qua một locator; (2) frontend phát triển và cập nhật giao diện theo từng sprint; (3) sau mỗi thay đổi, CI tự động chạy lại bộ kiểm thử; (4) test runner thực thi từng bước, định vị phần tử bằng locator rồi mô phỏng thao tác người dùng; (5) kết quả (đạt/không đạt) được trả về cho QA. Trong trường hợp một locator không còn khớp phần tử do giao diện đã thay đổi, bước (4) phát sinh NoSuchElementException; QA xác định nguyên nhân và cập nhật lại locator trước khi chạy lại chu trình.

3.2 Kiến trúc tổng thể hệ thống



Hình 3.1 Luồng hệ thống

Hệ thống self-healing được thiết kế theo mô hình pipeline gồm 5 module độc lập. Luồng xử lý chính như sau:

Bước	Module	Mô tả
1	find_element()	Test gọi find_element() như Selenium thông thường
2	Exception Interceptor	Phân loại exception và quyết định chiến lược
3	Snapshot Store	Load snapshot phần tử đã lưu từ lần chạy trước thành công
4	Similarity Engine V2	Tính điểm tương đồng 5 chiều cho mọi candidate
5	Candidate Ranker	Xếp hạng, chọn best candidate, quyết định AUTO_FIX/SUGGEST
6	Knowledge Base	Cache kết quả healing để tái dùng lần sau

Bảng 3.1 Luồng xử lý chính của hệ thống

Khi test chạy thành công (tìm được element), hệ thống tự động chụp snapshot và lưu vào SQLite. Khi test thất bại (NoSuchElementException), hệ thống kích hoạt pipeline healing, sử dụng snapshot đã lưu làm tham chiếu để tìm phần tử thay thế phù hợp nhất trong DOM hiện tại.

Weights: Hệ thống bắt đầu với `STATIC_WEIGHTS = {attribute: 0.27, semantic: 0.24, structure: 0.20, visual: 0.12, context: 0.17}`. Sau khi tích lũy đủ 100 healing events, tự động retrain Logistic Regression (pairwise) và chuyển sang `LEARNED weights`

3.3 Module 1 – Exception Interceptor

Exception Interceptor có nhiệm vụ bắt và phân loại các exception từ Selenium WebDriver. Không phải mọi exception đều cần self healing, nhiều loại có thể xử lý theo cách đơn giản hơn.

Loại Exception	Chiến lược xử lý	Kích hoạt Healing?
NoSuchElementException	TRIGGER_HEALING — kích hoạt pipeline tìm phần tử thay thế	Có

Loại Exception	Chiến lược xử lý	Kích hoạt Healing?
StaleElementReferenceException	RETRY_FIND — thử find lại với locator cũ	Không
TimeoutException	WAIT_AND_RETRY — chờ thêm 10 giây rồi thử lại	Không
ElementNotInteractableException	SCROLL_INTO_VIEW — cuộn đến phần tử	Không
ElementClickInterceptedException	JS_CLICK — dùng JavaScript thay Selenium	Không
Các exception khác	RAISE — không xử lý, báo lỗi cho kỹ sư	Không

Bảng 3.2 Phân loại các Exception

Thiết kế này đảm bảo self healing chỉ được kích hoạt đúng mục đích (locator lỗi do UI thay đổi), không lãng phí tài nguyên cho các lỗi tạm thời hoặc môi trường.

3.4 Module 2 – Snapshot Store

Khi Selenium tìm thấy một phần tử thành công, hệ thống tự động chụp 'snapshot' — một bản ghi đầy đủ các thuộc tính của phần tử đó tại thời điểm test chạy. Snapshot được lưu vào cả JSON file và SQLite database để đảm bảo tính bền vững.

Cấu trúc dữ liệu ElementSnapshot bao gồm các trường thông tin:

Nhóm	Trường	Mô tả
Định danh	id, data_testid, name	Các thuộc tính định danh chính của phần tử
Thuộc tính	tag, classes, input_type, placeholder, aria_label	Các thuộc tính HTML của phần tử
Ngữ nghĩa	text, role, href, nearby_label, value	Nội dung và ngữ nghĩa của phần tử

Nhóm	Trường	Mô tả
Cấu trúc	xpath_abs, parent_tag, parent_id, parent_classes, depth, sibling_index, siblings_count	Vị trí trong cấu trúc DOM
Trực quan	bounding_x, bounding_y, bounding_w, bounding_h, viewport_w, viewport_h, is_visible, in_viewport, z_index, css_style_hash, computed_font_size, computed_color	Kích thước và vị trí trực quan
Ngữ cảnh	form_context, page_url, nearest_landmark, scroll_container_hash, position_in_form, preceding_text, modal_or_overlay, shadow_root_depth	Ngữ cảnh trang và form chứa phần tử
Meta	step_name, ui_version, captured_at, locator_type, locator_value	Thông tin kiểm thử

Bảng 3.3 Các trường thông tin trong ElementSnapshot

3.5 Module 3 – Similarity Engine

Đây là module cốt lõi của hệ thống. Với mỗi cặp (snapshot gốc, candidate DOM mới), engine tính điểm tương đồng theo 5 chiều, mỗi chiều có công thức riêng.

3.5.1 Chiều 1 – Attribute Score

Chiều Attribute đo mức độ tương đồng các thuộc tính HTML giữa phần tử gốc và candidate. Đây là chiều truyền thống nhất, thường được coi là quan trọng nhất trong các nghiên cứu trước.

Thuộc tính	Trọng số nội chiều	Ghi chú
id	0.29	Giảm 50% nếu phát hiện là dynamic value
classes	0.22	Dùng Jaccard Similarity cho tập CSS classes
name	0.15	SequenceMatcher ratio
input_type	0.10	So sánh chính xác (text, password, email, ...)
placeholder	0.09	SequenceMatcher — gợi ý chức năng của input
aria_label	0.08	SequenceMatcher — thông tin accessibility
data_testid	0.07	Giảm 70% nếu phát hiện là dynamic value

Bảng 3.4 Trọng số thuộc tính trong Attribute

Thuộc tính aria_label được đưa vào vì là thuộc tính định nghĩa theo chuẩn WAI-ARIA [10], cung cấp nhãn trợ năng ổn định ngay cả khi developer tái cấu trúc DOM.

Tag name (div, input, button, ...) đóng vai trò như nhân tử: nếu tag không khớp, toàn bộ điểm Attribute nhân với hệ số 0.5. Nếu tag khớp, nhân 1.0. Điều này đảm bảo không bao giờ nhầm lẫn giữa các phần tử thuộc loại khác nhau.

3.5.2 Chiều 2 – Semantic Score

Chiều Semantic đo tương đồng ngữ nghĩa - nội dung mà người dùng thực sự nhìn thấy và tương tác. Đây là chiều ít phụ thuộc vào cách implement kỹ thuật, do đó bền vững hơn khi developer tái cấu trúc code.

Đặc trưng	Trọng số nội chiều	Ý nghĩa
text	0.38	Nội dung văn bản hiển thị (innerText)
role	0.22	ARIA role (button, textbox, link, v.v.)
href	0.22	URL đích đến nếu là liên kết
nearby_label	0.13	Nhãn form gần nhất (label, legend)

Đặc trưng	Trọng số nội chiều	Ý nghĩa
value	0.05	Giá trị hiện tại của input/select

Bảng 3.5 Trọng số thuộc tính trong Semantic

Đặc trưng role theo chuẩn WAI-ARIA [10] phản ánh ngữ nghĩa của phần tử độc lập với tên thẻ HTML, giúp nhận diện phần tử khi developer đổi từ <button> sang <div role=\"button\">.

3.5.3 Chiều 3 – Structure Score

Chiều Structure đo vị trí của phần tử trong cấu trúc cây DOM. Ngay cả khi thuộc tính và văn bản thay đổi, cấu trúc cây thường ổn định hơn vì nó phản ánh thiết kế bố cục giao diện.

Đặc trưng	Trọng số	Ý nghĩa
xpath_abs	0.26	XPath tuyệt đối — chuỗi đường dẫn đầy đủ
parent_tag	0.22	Tag của phần tử cha (form, div, nav, v.v.)
parent_id	0.18	ID của phần tử cha
parent_classes	0.14	Tập CSS classes của phần tử cha (Jaccard)
depth	0.08	Độ sâu trong DOM (giảm theo $ \text{depth}_a - \text{depth}_b \times 0.25$)
sibling_index	0.07	Vị trí so với anh/chị em cùng cấp
siblings_count	0.05	Số anh/chị em cùng cấp

Bảng 3.6 Trọng số thuộc tính trong Structure

3.5.4 Chiều 4 – Visual Score

Chiều Visual đo tương đồng trực quan dựa trên bounding box — vùng hình chữ nhật bao quanh phần tử trên màn hình. Chiều này đặc biệt hiệu quả vì ngay cả khi mã nguồn thay đổi nhiều, phần tử thường vẫn giữ vị trí và kích thước tương tự trên màn hình.

Công thức tính Visual Score kết hợp điểm kích thước (size score) và điểm vị trí (position score):

$$\text{visual_score} = \text{bbox_size} \times 0.35 + \text{bbox_pos} \times 0.25 + \text{css_hash} \times 0.25 + \text{visibility} \times 0.15$$

Trong đó:

bbox_size và **bbox_pos** được tính dựa trên tọa độ tương đối so với viewport (thay vì giá trị pixel tuyệt đối), giúp kết quả ổn định khi chạy trên các màn hình có độ phân giải khác nhau:

$$\text{size_sim} = 1 - (|\text{rw}_1 - \text{rw}_2| + |\text{rh}_1 - \text{rh}_2|) / 2$$

$$\text{pos_sim} = 1 - \sqrt{((\text{rx}_1 - \text{rx}_2)^2 + (\text{ry}_1 - \text{ry}_2)^2) \times 3.0}$$

Với $\text{rw} = \text{bounding_w} / \text{viewport_w}$, $\text{rh} = \text{bounding_h} / \text{viewport_h}$, tương tự cho rx , ry . Nếu phần tử không có bounding box hợp lệ ($\text{bounding_w} = 0$), cả hai thành phần nhận giá trị neutral thấp 0.20 — thấp hơn so với neutral thông thường để phản ánh đây là dấu hiệu bất thường (phần tử ẩn hoặc ngoài màn hình).

css_hash so sánh fingerprint MD5 được tạo từ 16 computed CSS property ổn định (display, font-size, color, background-color, v.v.). Nếu hash trùng khớp, điểm là 1.0; nếu khác nhau, điểm phụ thuộc vào loại phần tử: 0.50 với form element (button, input, a) và 0.25 với các phần tử còn lại. Khi thiếu hash ở một trong hai phía, dùng neutral 0.30.

visibility tổng hợp ba yếu tố: trạng thái hiển thị (is_visible, trọng số 0.50), vị trí trong viewport (in_viewport, trọng số 0.30), và z-index ($0.20 \times \max(0, 1 - |z_1 - z_2| / 100)$). Mỗi yếu tố rơi về neutral tương ứng khi giá trị là None.

3.5.5 Chiều 5 – Context Score

Chiều Context đo tương đồng ngữ cảnh bao quanh phần tử, gồm sáu yếu tố với trọng số phân bổ như sau:

$$\text{context_score} = \text{position_in_form} \times 0.30 + \text{preceding_text} \times 0.25 + \text{font_size} \times 0.20 + \text{color} \times 0.15 + \text{landmark} \times 0.06 + \text{scroll} \times 0.04$$

Trong đó:

position_in_form: vị trí thứ tự của phần tử trong form (đếm tất cả input, select, textarea, button không ẩn). Cho điểm 1.0 nếu trùng khớp, giảm dần theo $1 - |\Delta \text{pos}| \times 0.3$ nếu lệch. Yếu tố này giúp phân biệt các field có thuộc tính giống nhau trong cùng một form (ví dụ: email field ở vị trí 0 so với password field ở vị trí 1). Neutral 0.50 khi không xác định được.

preceding_text: text node hoặc nội dung element liền trước — đóng vai trò label không chính thức khi phần tử không có <label> liên kết. So sánh bằng token set similarity.

font_size và **color**: lấy từ computed style, giúp phân biệt primary action với secondary action hoặc heading với body text. Cho điểm 1.0 nếu trùng, 0.20/0.30 nếu khác, neutral 0.50 khi thiếu.

landmark: ARIA landmark gần nhất (main, nav, dialog, v.v.) — giúp xác định phần tử thuộc vùng nào trên trang.

scroll: fingerprint của scroll container gần nhất — giúp phân biệt các phần tử nằm trong các danh sách cuộn khác nhau.

3.5.6 Công thức tổng hợp SAW

Điểm tương đồng tổng hợp được tính theo công thức SAW [15]:

$$\text{total_score} = w_attr \times attr + w_sem \times sem + w_struct \times struct + w_vis \times vis + w_ctx \times ctx$$

Trong đó:

- + w_attr : trọng số của attribute /attr: attribute score
- + w_sem : trọng số của semantic /sem: semantic score
- + w_struct : trọng số của structure/structr: structure score
- + w_vis : trọng số của visual/vis: visual score
- + w_ctx : trọng số của context/ctxr: context score

Các bộ trọng số w_attr , w_sem , w_struct , w_vis , w_ctx được tính theo quá trình SAW [15] như sau:

1. Xác định tiêu chí đánh giá

Kí hiệu	Tiêu chí	Ý nghĩa
C1	Độ ổn định (Stability)	Mức độ thay đổi ít khi giao diện thay đổi
C2	Độ chính xác (Accuracy)	Khả năng xác định đúng phần tử
C3	Khả năng chống lỗi mutation (Mutation Resistance)	Chịu được rename/remove/replace
C4	Chi phí tính toán (Computation Cost)	Tài nguyên xử lý cần thiết

Bảng 3.7 Tiêu chí đánh giá

Trong đó:

- + C1, C2, C3 là Benefit criteria (càng lớn càng tốt)
- + C4 là Cost criteria (càng nhỏ càng tốt)

2. Xác định tỷ lệ phù hợp cho từng phương án

Chuyên gia đánh giá mức độ phù hợp theo thang điểm:

Điểm	Mức độ
1	Rất thấp
2	Thấp
3	Trung bình
4	Cao
5	Rất cao

Bảng 3.8 Điểm mức độ phù hợp

Sau khi phân tích thực nghiệm hệ thống self healing, ta có bảng đánh giá các tiêu chí:

Similarity Dimension	C1	C2	C3	C4
Attribute	5	5	4	2
Semantic	4	4	5	3
Structure	4	3	4	4
Context	3	3	3	3
Visual	2	2	2	5

Bảng 3.9 Đánh giá các tiêu chí

Giải thích:

+ Attribute:

- Nhiều thuộc tính như id, name, data-testid thường được dev giữ ổn định để phục vụ automation, ít thay đổi → ổn định
- Thường xác định trực tiếp đúng phần tử → độ chính xác cao
- Nếu id bị thay đổi thì các thuộc tính khác như class, type, name vẫn có thể hỗ trợ healing. Nhưng nếu attribute bị remove hoàn toàn thì hiệu quả giảm → không phải tuyệt đối.
- So sánh string nhanh, nhẹ, ít tài nguyên → chi phí thấp

+ Semantic:

- Text có thể thay đổi nhưng ý nghĩa vẫn gần nhau → khá ổn định
- Nhiều trường hợp xác định tốt nhưng không bằng attribute
- Semantic mạnh với mutation khi xảy ra rename, text replace, localization VD: login → sign in vẫn heal được
- Cần NLP, embedding, cosine similarity nặng hơn string compare → cost trung bình

+ Structure:

- DOM thường khá ổn định, nhưng nếu frontend refactor như div → section thì thay đổi mạnh → cao nhưng không tối đa
- Nhiều phần tử có structure giống nhau VD: 10 nút button trong form → dễ ambiguity → trung bình

- Khi attribute biến mất, structure vẫn giúp heal tốt → khá mạnh
- Phải parse DOM tree, compare Xpath, tree matching → tốn tài nguyên hơn

+ Context:

- Context thay đổi khá thường xuyên VD: thêm banner mới → context thay đổi → trung bình
- Không trực tiếp xác định phần tử → trung bình
- Khi layout thay đổi, context dễ sai → trung bình
- Cần phân tích vùng lân cận → trung bình

+ Visual:

- Layout thay đổi liên tục → rất không ổn định → thấp
- Nhiều phần tử giống nhau VD: nhiều button màu xanh → dễ nhầm → thấp
- Thay đổi chủ đề → visual hỏng → thấp
- Cần screenshot, image processing, CV model → cực nặng → rất cao

3. Lập ma trận quyết định

$$X = \begin{bmatrix} 5 & 5 & 4 & 2 \\ 4 & 4 & 5 & 3 \\ 4 & 3 & 4 & 4 \\ 3 & 3 & 3 & 3 \\ 2 & 2 & 2 & 5 \end{bmatrix}$$

Trong đó:

- + Hàng = phương án (5 chiều similary)
- + Cột = tiêu chí đánh giá

4. Chuẩn hóa ma trận

Công thức chuẩn hóa:

Với Benefit criteria:

$$r_{ij} = \frac{x_{ij}}{\max(x_{ij})}$$

Với Cost criteria:

$$r_{ij} = \frac{\min(x_{ij})}{x_{ij}}$$

Ma trận sau khi chuẩn hóa

$$X = \begin{bmatrix} 1 & 1 & 0.8 & 1 \\ 0.8 & 0.8 & 1 & 0.67 \\ 0.8 & 0.6 & 0.8 & 0.5 \\ 0.6 & 0.6 & 0.6 & 0.67 \\ 0.4 & 0.4 & 0.4 & 0.4 \end{bmatrix}$$

Similarity Dimension	C1	C2	C3	C4
Attribute	1	1	0.8	1
Semantic	0.8	0.8	1	0.67
Structue	0.8	0.6	0.8	0.5
Context	0.6	0.6	0.6	0.67
Visual	0.4	0.4	0.4	0.4

Bảng 3.10 Hệ số sau khi chuẩn hóa của các chiều

5. Tính giá trị cuối cùng

Áp dụng phương pháp AHP (Analytic Hierarchy Process)_quy trình phân cấp phân tích để xác định trọng số tiêu chí [7].

B1: Lập ma trận so sánh cặp

Dựa trên đánh giá chuyên gia trong hệ thống self-healing:

- Stability quan trọng ngang Accuracy
- Stability quan trọng hơn Mutation Resistance
- Stability quan trọng hơn nhiều so với Cost
- Accuracy quan trọng hơn Mutation Resistance
- Accuracy quan trọng hơn nhiều so với Cost
- Mutation Resistance quan trọng hơn Cost

Sử dụng thang Saaty:

Giá trị	Ý nghĩa
1	Quan trọng như nhau
3	Hơi quan trọng hơn
5	Quan trọng hơn
7	Rất mạnh
9	Tuyệt đối

Bảng 3.11 Thang điểm Saaty

Bảng so sánh các tiêu chí theo thang điểm saaty:

	C1	C2	C3	C4
C1	1	1	3	5
C2	1	1	3	5
C3	1/3	1/3	1	3
C4	1/5	1/5	1/3	1

Bảng 3.12 So sánh các tiêu chí theo thang điểm Saaty

Ma trận AHP:

$$A = \begin{bmatrix} 1 & 1 & 3 & 5 \\ 1 & 1 & 3 & 5 \\ 1/3 & 1/3 & 1 & 3 \\ 1/5 & 1/5 & 1/3 & 1 \end{bmatrix}$$

B2: Tính tổng từng cột

$$\text{Cột 1} = 1+1+1/3+1/3 = 2.533$$

$$\text{Cột 2} = 1+1+1/3+1/3 = 2.533$$

$$\text{Cột 3} = 3+3+1+1/3 = 7.333$$

$$\text{Cột 4} = 5+5+3+1 = 14$$

B3: Chuẩn hóa ma trận

Công thức:

$$a_{ij}^{norm} = \frac{a_{ij}}{\sum column_j}$$

Ma trận sau khi chuẩn hóa

$$A = \begin{bmatrix} 0.395 & 0.395 & 0.409 & 0.357 \\ 0.396 & 0.395 & 0.409 & 0.357 \\ 0.132 & 0.132 & 0.136 & 0.214 \\ 0.079 & 0.079 & 0.045 & 0.071 \end{bmatrix}$$

Bảng so sánh tiêu chí sau khi chuẩn hóa:

	C1	C2	C3	C4
C1	0.395	0.395	0.409	0.357
C2	0.395	0.395	0.409	0.357
C3	0.132	0.132	0.136	0.214
C4	0.079	0.079	0.045	0.071

Bảng 3.13 So sánh các tiêu chí sau khi chuẩn hóa

B4: Tính vector trọng số

Lấy trung bình từng hàng:

$$+ \text{ Stability: } W_1 = \frac{0.394+0.395+0.409+0.409}{4} = 0.389 \approx 0.39$$

$$+ \text{ Accuracy: } W_2 = 0.389 \approx 0.39$$

$$+ \text{ Mutation Resistance: } W_3 = 0.154 \approx 0.15$$

$$+ \text{ Cost: } W_4 = 0.069 \approx 0.07$$

Tính SAW:

Công thức:

$$V_i = \sum_{j=1}^n r_{ij} \cdot w_j$$

Tính cho từng chiều:

$$+ \text{ Attribute: } V_1 = 1(0.39) + 1(0.39) + 0.8(0.15) + 1(0.07) = 0.97$$

$$+ \text{ Semantic: } V_2 = 0.8209$$

- + Structure: $V_3 = 0.701$
- + Context: $V_4 = 0.6049$
- + Visual; $V_5 = 0.4$

$$\sum V_i = 3.4968$$

Chuẩn hóa về tổng =1:

Công thức:

$$W = \frac{V_i}{\sum V_i}$$

- ➔ $w_{attr} = 0.27$
- ➔ $w_{sem} = 0.24$
- ➔ $w_{struct} = 0.2$
- ➔ $w_{vis} = 0.12$
- ➔ $w_{ctx} = 0.17$

Những bước này đều áp dụng cho các trọng số của từng thành phần trong mỗi feature.

3.6 Module 4 – Candidate Ranker

Sau khi Similarity Engine tính điểm cho tất cả các candidate trong DOM hiện tại, Candidate Ranker xếp hạng và đưa ra quyết định dựa trên ngưỡng điểm:

Điều kiện	Quyết định	Hành động
$score \geq 0.7$	AUTO_FIX	Tự động dùng candidate này, tiếp tục chạy test
$0.50 \leq score < 0.7$	SUGGEST	Đề xuất candidate nhưng cần xác nhận thủ công
$score < 0.50$ hoặc không có candidate	FAILED	Không tìm được phần tử phù hợp, báo lỗi

Bảng 3.14 Ngưỡng điểm quyết định

Khi AUTO_FIX được kích hoạt, hệ thống còn chọn locator bền vững nhất cho lần sau theo thứ tự ưu tiên: data-testid > aria-label > name > placeholder > id > text (cho button/link). Locator này được lưu vào Knowledge Base.

3.7 Module 5 – Knowledge Base

Knowledge Base là cơ chế cache kết quả healing để tăng tốc các lần test tiếp theo. Sau mỗi lần healing thành công, hệ thống lưu:

- `step_name`: tên bước test (dùng làm khóa tra cứu)
- `old_locator`: locator cũ đã thất bại
- `new_locator`: locator mới hoạt động tốt
- `confidence`: số điểm locator mới đạt được
- `similarity_detail`: chi tiết điểm tương đồng
- `ui_version`: phiên bản UI tại thời điểm healing
- `time_used`: số lần đã dùng locator này thành công

Khi test bắt đầu chạy và gặp lỗi, hệ thống kiểm tra Knowledge Base trước khi chạy full healing pipeline. Nếu tìm thấy locator đã được cache và vẫn hoạt động, hệ thống dùng luôn mà không cần tính toán lại similarity — giảm thời gian healing từ ~2 giây xuống còn ~50ms.

Knowledge Base được lưu dưới dạng JSON file trong thư mục `knowledge_base/`, dễ dàng inspect và chỉnh sửa thủ công nếu cần.

3.8 Thiết kế cơ chế tối ưu trọng số bằng Logistic Regression

3.8.1 Chuẩn bị dữ liệu

Dữ liệu huấn luyện được đọc từ bảng `candidate_scores` trong SQLite database, mỗi hàng là một cặp (`healing_event_id`, `candidate_element`) gồm 5 sub-score và nhãn `is_correct`. Hệ thống áp dụng Pairwise Learning to Rank [5] để tối ưu hóa trọng số 5 chiều đặc trưng. Thay vì huấn luyện trực tiếp trên từng điểm số (pointwise), hệ thống học từ sự tương phản giữa các cặp candidate trong cùng một healing event.

Với mỗi event có winner (`is_correct=1`) và losers (`is_correct=0`), mọi cặp (`w`, `l`) được tạo ra:

- $\text{diff} = w_feats - l_feats \rightarrow \text{nhãn } y = 1$
- $-\text{diff} (\text{mirror}) \rightarrow \text{nhãn } y = 0$

Cách tạo mirror sample này đảm bảo tập huấn luyện luôn cân bằng nhãn theo cặp. Các cặp có $|\text{diff}|\text{.sum} < 1e-6$ (hai candidate gần như giống hệt nhau) bị loại bỏ vì là nhiễu. Các event thiếu một phía (toàn winner hoặc toàn loser) cũng bị bỏ qua. Hệ thống chỉ bắt

đầu retrain lần đầu sau khi tích lũy đủ 100 healing thành công, và retrain lại sau mỗi 20 healing tiếp theo

Cách tiếp cận này phù hợp vì bài toán thực chất là ranking không phải phân loại, trong đó mục tiêu là xác định candidate nào giống original nhất trong tập ứng viên của từng event.

Vì sao lại chọn phương pháp này?

Ví dụ:

- Heal Event 1 (UI thay đổi ít):
 - Phần tử đúng ($A = [0.8, 0.5, 0.7, 0.9, 0.6]$) (Nhãn: Đúng)
 - Phần tử sai ($B = [0.7, 0.6, 0.5, 0.8, 0.6]$) (Nhãn: Sai)
- Heal Event 2 (UI thay đổi nhiều):
 - Phần tử đúng ($C = [0.6, 0.4, 0.5, 0.7, 0.5]$) (Nhãn: Đúng)
 - Phần tử sai ($D = [0.5, 0.4, 0.3, 0.7, 0.4]$) (Nhãn: Sai)
- Nếu nạp trực tiếp dữ liệu thô trên vào mô hình để tìm ngưỡng phân loại Đúng/Sai tuyệt đối, mô hình sẽ gặp hiện tượng xung đột dữ liệu:
- Tại chiều số 1 (Attribute): Điểm (0.7) (của B)) bị gán nhãn là Sai, nhưng điểm (0.6) (của C)) thấp hơn lại được gán nhãn là Đúng.
- Tại chiều số 3 (Structure): Điểm (0.5) (của B)) là Sai, nhưng điểm (0.5) (của C)) lại là Đúng.

Sự mâu thuẫn này khiến mô hình bị hỗn loạn, không thể tìm được ranh giới quyết định (Decision Boundary) đồng nhất. Trọng số học được sẽ bị sai lệch hoàn toàn.

Cách phương pháp Pairwise giải quyết vấn đề

Bằng cách áp dụng phép toán hiệu số nội bộ ($\Delta X = X_{\text{đúng}} - X_{\text{sai}}$), dữ liệu lập tức được chuyển sang hệ quy chiếu chênh lệch tương đối:

- Từ Heal Event 1 (Lấy $A - B$): $\Delta X_1 = [+0.1, -0.1, +0.2, +0.1, 0.0]$
- Từ Heal Event 2 (Lấy $C - D$): $\Delta X_2 = [+0.1, 0.0, +0.2, 0.0, +0.1]$

Kết luận: Sau khi biến đổi, sự mâu thuẫn dữ liệu ban đầu hoàn toàn bị triệt tiêu. Mô hình dễ dàng tìm ra mẫu số chung ổn định ở các chiều cốt lõi (như chiều số 1 luôn chênh lệch (+0.1) và chiều số 3 luôn chênh lệch (+0.2)). Phương pháp Pairwise đã chuyển bài toán từ việc "tìm một phần tử hoàn hảo" sang "tìm phần tử tốt nhất trong các lựa chọn hiện có", giúp mô hình học được bộ trọng số tối ưu một cách nhất quán và chính xác.

3.8.2 *Phân chia dữ liệu*

Do thiết kế pairwise, mỗi mẫu trong tập huấn luyện là một hiệu vector giữa hai candidate trong cùng một event — không còn mang định danh event gốc. Vì vậy, nguy cơ data leakage do các candidate của cùng một event bị phân tán vào train và test được giải quyết ngay từ bước xây dựng dữ liệu: mỗi cặp pairwise chỉ thuộc về một event duy nhất. Hệ thống không áp dụng GroupShuffleSplit để chia train/test mà huấn luyện trực tiếp trên toàn bộ pairwise samples, phù hợp với mục tiêu là cập nhật trọng số liên tục theo thời gian thực trong vòng đời test automation.

3.8.3 *Huấn luyện mô hình*

Mô hình được triển khai bằng thư viện scikit-learn [9] sử dụng LogisticRegression với các tham số: `penalty='l2'`, `C=1.0`, `max_iter=1000`, `solver='lbfgs'`, `class_weight='balanced'`, `random_state=42`. Không cần chuẩn hóa feature vì tất cả sub-score đã nằm trong khoảng $[0, 1]$. Hệ thống kiểm tra tối thiểu 20 pairwise samples và sự tồn tại của cả hai class trước khi train; nếu không đủ điều kiện, trọng số mặc định (DEFAULT_WEIGHTS) được giữ nguyên.

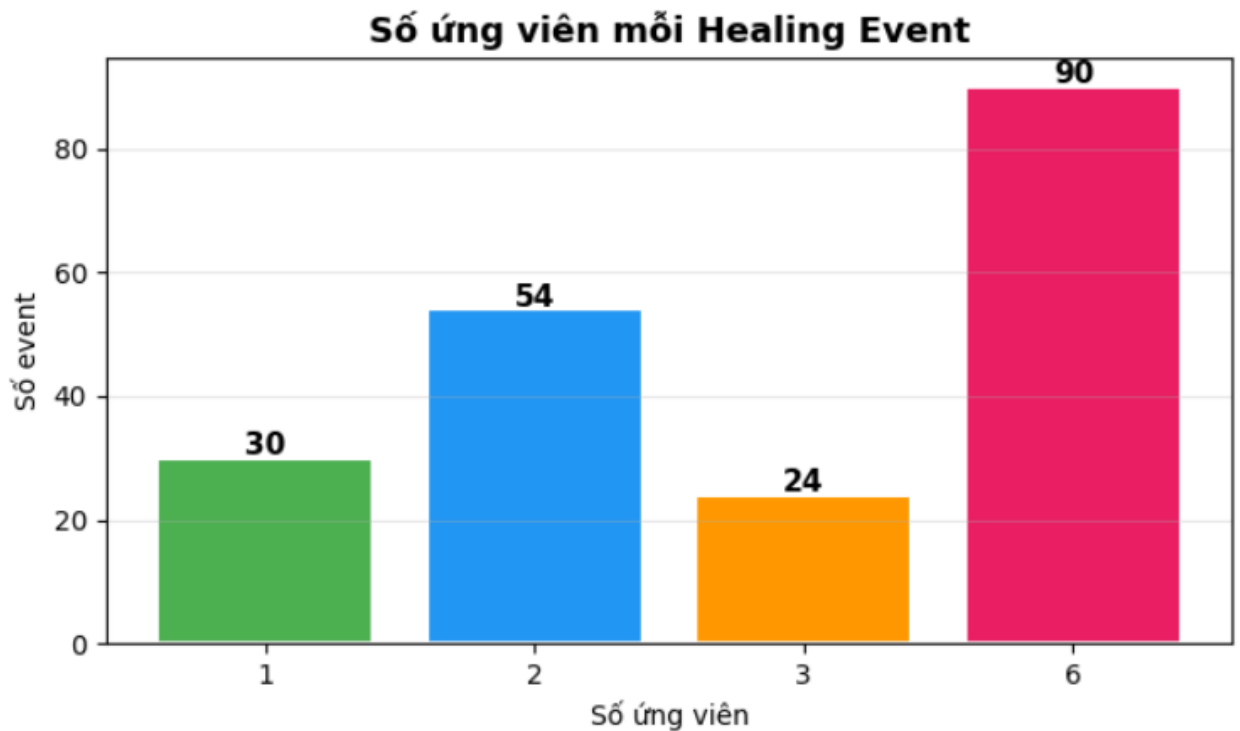
3.8.4 *Chuyển đổi sang trọng số tối ưu*

Sau khi train, hệ số `coef_[0]` của model được chuyển thành trọng số tối ưu qua hai bước: (1) lấy giá trị tuyệt đối `|coef|` để loại bỏ dấu âm (vì pairwise diff có thể cho coef âm tùy chiều), (2) chuẩn hóa để tổng bằng 1. Kết quả được lưu vào bảng `learned_weights` trong database kèm metadata (`trained_at`, `n_samples`, `accuracy`, `notes`) để phục vụ audit. Ở lần khởi động tiếp theo, hệ thống tự động tải bộ trọng số mới nhất từ DB thay vì dùng DEFAULT_WEIGHTS.

3.9 *Thiết kế ngưỡng quyết định*

3.9.1 *Bộ dữ liệu phân tích*

Phân tích được thực hiện trên 750 bản ghi từ 198 healing event, trong đó mỗi event có đúng 1 ứng viên đúng (`is_correct=1`) và trung bình 4 ứng viên sai — tỷ lệ lớp mất cân bằng 1:4 (552/750 bản ghi là ứng viên sai). Mỗi bản ghi bao gồm 5 sub-score và `total_score` được tính theo bộ trọng số tĩnh. Bộ dữ liệu bao phủ 11 phiên bản UI từ V1 đến V11, với mức độ thay đổi từ nhẹ (chỉ một chiều) đến nặng (ba chiều đồng thời).



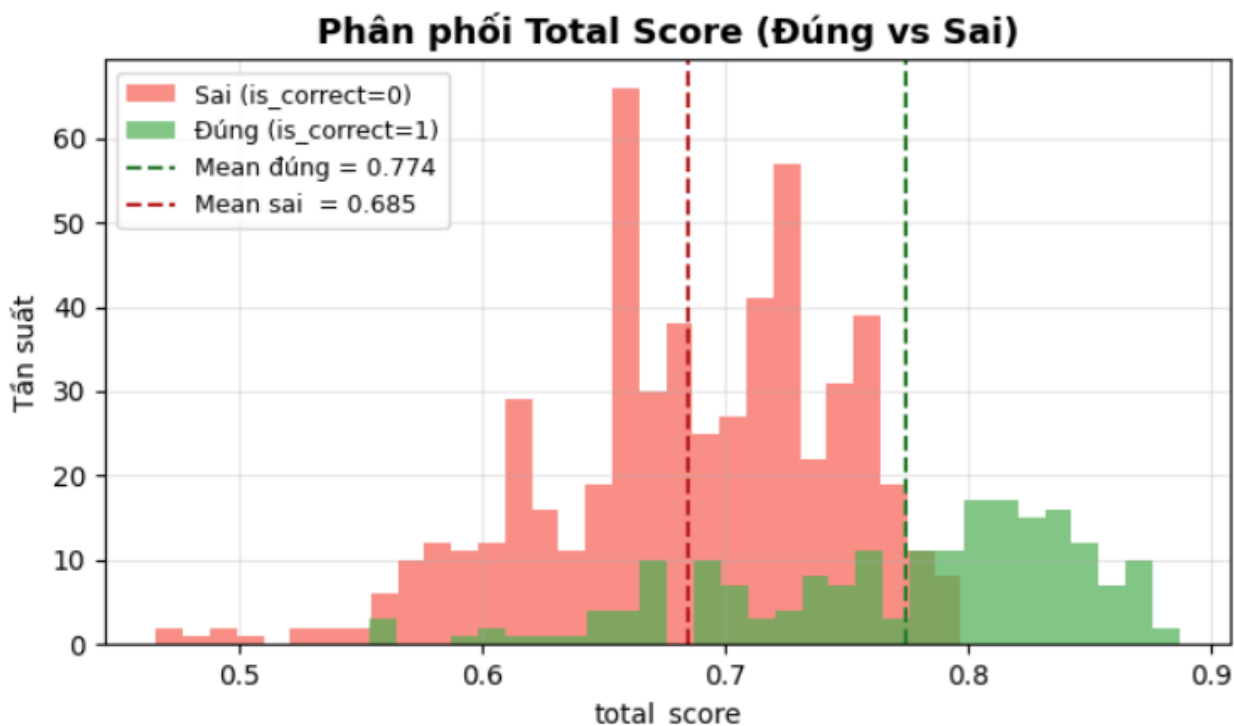
Hình 3.2 Phân phối số ứng viên của các event

3.9.2 Phân tích khả năng phân tách

Bước đầu tiên là xác minh xem `total_score` có đủ khả năng phân biệt ứng viên đúng với ứng viên sai hay không. Với mỗi event, hệ thống kiểm tra liệu $\text{score}(\text{đúng}) > \max(\text{score}(\text{sai}))$ có thành lập không, đồng thời tính biên độ cách biệt $\text{margin} = \text{score}(\text{đúng}) - \max(\text{score}(\text{sai}))$.

Kết quả: **100% trong số 198 event đều separable** — ứng viên đúng luôn xếp hạng 1. Biên độ trung bình đạt 0.2263, phân phối lệch phải mạnh với phần lớn event có cách biệt 0.20–1.00. Tuy nhiên còn 2 event có margin dưới 0.01 (nhỏ nhất là 0.0014) — đây là những trường hợp sát nút cần giám sát khi DOM tiếp tục thay đổi

3.9.3 Phân tích phân phối điểm số



Hình 3.3 Phân phối điểm số 1

Phân phối `total_score` giữa hai nhóm cho thấy:

Ứng viên đúng: mean = 0.775, tập trung trong khoảng 0.70–0.89; best score thấp nhất toàn bộ dataset là **0.5537**.

Ứng viên sai: mean = 0.685, phân phối rải rộng hơn và chồng lấp với nhóm đúng trong vùng 0.60–0.75.

Khoảng cách trung bình giữa hai nhóm (~ 0.090) đủ để phân tách hiệu quả nếu chọn ngưỡng phù hợp. Đáng chú ý là không có element đúng nào có best score dưới 0.55, trong đó 40 event nằm trong khoảng 0.55–0.69 và 158 event từ 0.70 trở lên.

3.9.4 Lựa chọn ngưỡng

Grid search được thực hiện trên không gian (`low_threshold`, `high_threshold`) với ràng buộc $\text{gap} \geq 0.10$ giữa hai ngưỡng. Các bộ ngưỡng được đánh giá theo thứ tự ưu tiên:

False positive = 0 — AI tự heal sai là hậu quả nghiêm trọng nhất, ước tính chi phí xử lý gấp $15\times$ so với bỏ qua.

Fail rate = 0% — không bỏ sót bất kỳ healing event nào có thể cứu được.

`high_threshold` ≥ 0.70 — chỉ tự động hóa khi model thực sự tự tin.

`low_threshold` ≥ 0.50 — đảm bảo lưới đỡ thực tế dựa trên sàn điểm quan sát được (0.5537).

Tối thiểu hóa tỷ lệ human review — trong số các bộ thỏa mãn 4 điều kiện trên, chọn bộ có auto rate cao nhất.

	low	high	gap	auto_rate (%)	human_rate (%)	fail_rate (%)	false_pos	cost
0	0.45	0.55	0.10	100.0	0.0	0.0	0	0
1	0.45	0.60	0.15	98.0	2.0	0.0	0	4
9	0.50	0.60	0.10	98.0	2.0	0.0	0	4
2	0.45	0.65	0.20	94.4	5.6	0.0	0	11
10	0.50	0.65	0.15	94.4	5.6	0.0	0	11
17	0.55	0.65	0.10	94.4	5.6	0.0	0	11
44	0.85	0.95	0.10	0.0	13.1	86.9	0	26
24	0.60	0.70	0.10	79.8	18.2	2.0	0	36
3	0.45	0.70	0.25	79.8	20.2	0.0	0	40
11	0.50	0.70	0.20	79.8	20.2	0.0	0	40
18	0.55	0.70	0.15	79.8	20.2	0.0	0	40
30	0.65	0.75	0.10	67.2	27.3	5.6	0	54
25	0.60	0.75	0.15	67.2	30.8	2.0	0	61
4	0.45	0.75	0.30	67.2	32.8	0.0	0	65
12	0.50	0.75	0.25	67.2	32.8	0.0	0	65

Hình 3.4 Top 15 bộ ngưỡng tối ưu

Bộ ngưỡng tối ưu được chọn là low = 0.50 và high = 0.70, cho kết quả:

Vùng	Điều kiện	Số event	Tỷ lệ
Auto heal	$\text{score} \geq 0.70$	158	79.8%
Human review	$0.50 \leq \text{score} < 0.70$	40	20.2%
Fail / Báo lỗi	$\text{score} < 0.50$	0	0.0%

Các bộ ngưỡng thấp hơn như (0.45, 0.55) hoặc (0.50, 0.60) về mặt kỹ thuật cũng đạt zero FP và zero fail do sàn điểm thực tế là 0.5537, nhưng bị loại vì high_threshold < 0.70 không đủ tin cậy cho môi trường production.

3.10 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng SQLite làm backend lưu trữ, với thiết kế gồm bốn bảng phục vụ các mục đích khác nhau trong vòng đời healing.

Bảng `healing_events` ghi lại mỗi lần hệ thống phát hiện locator bị hỏng và khởi động quá trình tìm kiếm ứng viên. Mỗi hàng tương ứng với một cặp (`step_name`, `ui_version`) tại một thời điểm cụ thể, kèm quyết định cuối cùng (`auto/suggest/fail`) và locator mới được chọn.

Bảng `candidate_scores` lưu điểm số chi tiết của tất cả candidate được xét trong một healing event, bao gồm 5 sub-score, `total_score`, và nhãn `is_correct`. Đây là bảng trung tâm cung cấp dữ liệu huấn luyện cho `LogisticWeightModel` — query yêu cầu `healing_event_id` IS NOT NULL và đủ cả 5 sub-score để đảm bảo mẫu hợp lệ.

Bảng `learned_weights` lưu lịch sử các lần retrain, mỗi hàng ghi lại bộ trọng số mới cùng metadata (`trained_at`, `n_samples`, `accuracy`, `notes`). Hệ thống luôn đọc hàng có id lớn nhất làm trọng số hiện hành; nếu bảng rỗng, `DEFAULT_WEIGHTS` được dùng thay thế. Thiết kế append-only này cho phép rollback về bất kỳ phiên bản trọng số nào trong lịch sử.

Bảng `snapshots` (tương ứng với `SnapshotStore`) lưu bản ghi đầy đủ `ElementSnapshot` dưới dạng JSON cho mỗi bước test tại mỗi phiên bản UI. File được đặt tên theo pattern `{step_name}_{ui_version}_{fingerprint_hash}.json`, với bản sao lịch sử có timestamp trong thư mục `con history/` để phục vụ audit theo thời gian.

CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Môi trường thực nghiệm

Toàn bộ thực nghiệm được thực hiện trong môi trường cụ thể như sau:

Thành phần	Cấu hình / Phiên bản
Hệ điều hành	Windows 11
Python	3.14
Selenium WebDriver	4.x
ChromeDriver	Tương ứng Chrome phiên bản mới nhất
React.js (ứng dụng kiểm thử)	19.x (SPA — Single Page Application)
Node.js	22.x (chạy ứng dụng React)
SQLite	3.x (lưu trữ dữ liệu healing)
scikit-learn	1.x (huấn luyện Logistic Regression)
pandas, numpy	Xử lý và phân tích dữ liệu
matplotlib	Trực quan hóa kết quả
pytest	Framework chạy test tự động

Bảng 4.1 Môi trường thực nghiệm

4.2 Ứng dụng web thực nghiệm

Ứng dụng demo được xây dựng là một hệ thống giới thiệu sản phẩm đơn giản bao gồm các màn hình: Đăng nhập (Login), Đăng ký (Register), Danh sách sản phẩm (Products), Thêm sản phẩm (AddProduct) và liên hệ (Contact).

Ứng dụng được thiết kế với đầy đủ thuộc tính định danh (data-testid, id, name, aria-label) ở phiên bản v1 (baseline), sau đó được biến đổi để tạo ra các phiên bản v2-v11.

4.3 Bộ dữ liệu thực nghiệm

Dữ liệu thực nghiệm được thu thập trong quá trình chạy test tự động trên các phiên bản mutation. Cấu trúc bộ dữ liệu:

Bộ dữ liệu	Số lượng	Mô tả
Healing events	198	Số sự kiện healing (test bị hỏng và hệ thống tìm được candidate)
Candidate records	750	Tổng số cặp (event, candidate) với đầy đủ 5 sub-scores
Positive (is_correct=1)	198	Các candidate đúng (1 per event)
Negative (is_correct=0)	552	Các candidate sai
Tỷ lệ imbalance	~1:4	Mỗi event trung bình có 4 candidate sai
UI versions covered	v2 đến v11	10 phiên bản mutation khác nhau

Bảng 4.2 Bộ dữ liệu thực nghiệm

4.4 Các phiên bản mutation UI

Phiên bản	Loại mutation
v1	Baseline
v2	Semantic only (+testid đổi)
v3	Structure only (+testid đổi)
v4	Visual only (+testid đổi)
v5	Context only (+testid đổi)
v6	Attribute + Semantic
v7	Structure + Visual (+testid đổi)
v8	Attribute + Context

v9	Semantic + Structure (+testid đổi)
v10	Attribute + Semantic + Structure
v11	Visual + Context + Attribute

Bảng 4.3 Các phiên bản mutation UI

4.5 Chỉ số đánh giá

Do đặc điểm của bài toán, các chỉ số đánh giá được chọn lọc như sau:

- Top-1 Success Rate: Chỉ số quan trọng nhất. Tỷ lệ healing events mà candidate đúng được xếp hạng cao nhất. Đây là metric phản ánh trực tiếp hiệu quả thực tế của hệ thống.
- Score Gap trung bình: Hiệu số trung bình giữa điểm candidate đúng và điểm candidate sai cao nhất trong cùng event. Gap lớn = hệ thống tự tin hơn và ít rủi ro chọn sai khi dữ liệu nhiễu.
- Score Gap nhỏ nhất: Trường hợp xấu nhất — event có gap nhỏ nhất. Phản ánh 'điểm yếu' của hệ thống.
- Số sự kiện sát nút ($gap < 0.05$): Số event mà candidate đúng chỉ hơn candidate sai tốt nhất chưa đến 5 điểm. Đây là những event dễ chọn sai nhất nếu có nhiễu dữ liệu.

4.6 Huấn luyện mô hình tối ưu trọng số

4.6.1 Dataset sử dụng

Dữ liệu huấn luyện được đọc trực tiếp từ bảng candidate_scores trong SQLite database — được tích lũy tự động trong quá trình hệ thống self-healing chạy. Mỗi hàng là một cặp (healing_event_id, candidate_element) gồm 5 sub-score và nhãn is_correct. Tại thời điểm train lần đầu, bảng này chứa **388 candidates từ 100 healing event** (100 positive, 288 negative), tỷ lệ mất cân bằng ~1:3. Sau lần train đầu, hệ thống tự động retrain sau mỗi 20 healing event tích lũy thêm (RETRAIN_EVERY = 20), sử dụng toàn bộ dữ liệu hiện có trong DB tại thời điểm đó.

Cột	Kiểu	Ý nghĩa
attr_score	float [0,1]	Điểm tương đồng thuộc tính HTML (id, class, name, data-*)
sem_score	float [0,1]	Điểm tương đồng ngữ nghĩa văn bản hiển thị

Cột	Kiểu	Ý nghĩa
struct_score	float [0,1]	Điểm tương đồng cấu trúc DOM xung quanh phần tử
visual_score	float [0,1]	Điểm tương đồng vị trí và kích thước trực quan
ctx_score	float [0,1]	Điểm tương đồng ngữ cảnh DOM lân cận (cha, anh em)
is_correct	int {0,1}	Nhãn: 1 = candidate đúng, 0 = candidate sai

Bảng 4.4 Mô tả thuộc tính dữ liệu

Trước khi đưa vào huấn luyện, hệ thống lọc bỏ các hàng có bất kỳ sub-score nào là NULL hoặc thiếu healing_event_id. Không có bước chuẩn hóa feature vì tất cả 5 sub-score đã nằm trong khoảng [0, 1]. Cột total_score bị loại bỏ có chủ đích — đây là tổng tuyến tính có trọng số của 5 sub-score và chính là đại lượng mà hệ thống cần tối ưu; đưa nó vào feature sẽ khiến model học nhãn thông qua kết quả nó được yêu cầu tối ưu, dẫn đến overfitting nghiêm trọng.

4.6.2 Xây dựng dữ liệu huấn luyện

Thay vì học trực tiếp $P(\text{is_correct}=1 \mid \text{features})$ trên các mẫu đơn lẻ, hệ thống áp dụng **pairwise learning**: model học $P(\text{winner} > \text{loser} \mid \text{winner_feats} - \text{loser_feats})$ trong cùng một healing event. Cách tiếp cận này phù hợp hơn với mục tiêu thực tế — xếp hạng candidate đúng cao hơn candidate sai, không phải phân loại nhị phân độc lập.

Quy trình xây dựng X và y từ raw data như sau: với mỗi healing event, hệ thống gom toàn bộ candidate vào hai nhóm winners ($\text{is_correct}=1$) và losers ($\text{is_correct}=0$). Sau đó tạo mọi cặp (w, l) có thể:

$\text{diff} = \text{w_feats} - \text{l_feats} \rightarrow$ thêm vào X, nhãn $y = 1$ (winner thắng)

$-\text{diff} \text{ (mirror)} \rightarrow$ thêm vào X, nhãn $y = 0$

Mỗi cặp luôn sinh đúng 2 mẫu đối xứng, đảm bảo tập huấn luyện cân bằng nhãn theo cặp mà không cần oversampling hay undersampling. Các cặp có $|\text{diff}|\text{.sum} < 1\text{e-}6$ bị loại bỏ vì hai candidate gần như giống hệt nhau — đây là nhiễu không mang thông tin phân biệt. Các event thiếu một phía (toàn winner hoặc toàn loser) cũng bị bỏ qua.

Từ 100 event đầu (388 candidates), quá trình này tạo ra **576 pairwise samples** — đây là X và y thực sự được đưa vào `lr.fit()`.

4.6.3 Lựa chọn mô hình

Tiêu chí	Logistic Regression	Neural Network	Deep Learning
Dữ liệu nhỏ	Hoạt động tốt	Overfit	Overfit
Dễ giải thích	Coefficient = trọng số trực tiếp	Black box	Black box sâu hơn
Tốc độ train	Milliseconds	Phút đến giờ	Chậm
Đưa ra trọng số tối ưu	Trực tiếp từ coef_	Cần thêm bước	Khó/không trực tiếp
Yêu cầu tài nguyên	Nhẹ (CPU đủ)	Trung bình	Nặng (thường cần GPU)

Bảng 4.5 So sánh lựa chọn model

Logistic Regression được chọn vì hệ số hồi quy coef_[0] sau khi lấy giá trị tuyệt đối và chuẩn hóa chính là bộ trọng số SAW cần tìm — không cần thêm bước diễn giải trung gian. Tham số class_weight='balanced' xử lý mất cân bằng nhãn trong pairwise data khi phân phối winner/loser không đều nhau giữa các event.

4.6.4 Huấn luyện và tính trọng số tối ưu

Model được khởi tạo với penalty='l2', C=1.0, max_iter=1000, solver='lbfgs', class_weight='balanced', random_state=42, rồi gọi lr.fit(X, y) trên toàn bộ 576 pairwise samples. Không có tập test riêng — accuracy = lr.score(X, y) được tính trên chính tập train và dùng làm chỉ số theo dõi chất lượng fit, không phải đánh giá tổng quát hóa. Hiệu quả thực tế của trọng số được đánh giá ở bước so sánh sau (mục 4.7).

Sau khi train, trọng số tối ưu được tính từ coef_[0] qua hai bước:

Bước 1: Lấy giá trị tuyệt đối |coef_i| — loại bỏ dấu âm (hệ số âm xuất hiện khi feature tương quan nghịch với nhãn trong không gian pairwise diff).

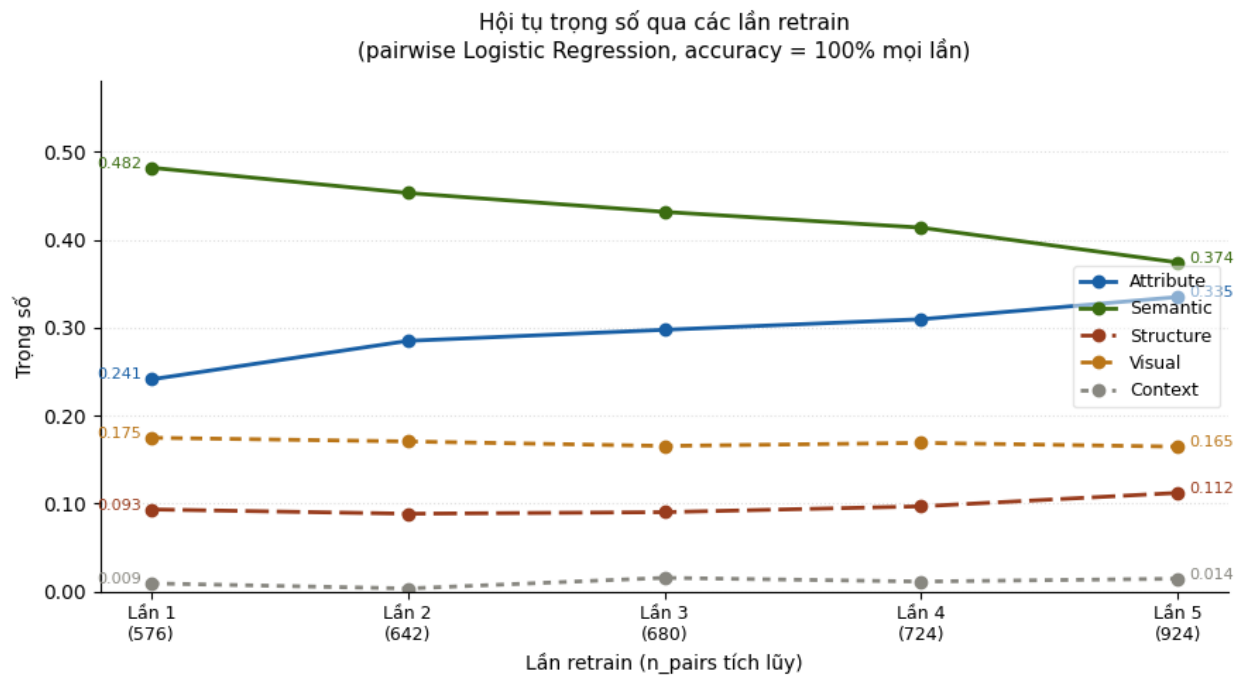
Bước 2: Chuẩn hóa: $w_i = |coef_i| / \sum |coef_j|$ — đảm bảo tổng bằng 1, tương thích với công thức SAW.

Kết quả lần train đầu tiên với (576 pairwise samples, accuracy = 100%):

Feature	SAW/AHP	Trọng số tối ưu	So với SAW/AHP
attribute	0.27	0.4819	+0.2419 ↑
semantic	0.24	0.2411	−0.0289 ↓
visual	0.2	0.1747	+0.0547 ↑
structure	0.12	0.0932	−0.1068 ↓
context	0.17	0.0091	−0.1609 ↓

Bảng 4.6 Kết quả train lần đầu

semantic trở thành chiều quan trọng nhất (0.4819), vượt xa SAW/AHP tính (0.24). Điều này phản ánh đặc điểm của dataset đang được sử dụng: text hiển thị là tín hiệu ổn định và phân biệt mạnh nhất trong các mutation — tương quan Pearson của sem_score với is_correct đạt 0.4769, cao nhất trong 5 chiều. context trọng số thấp nhất chỉ 0.0091.



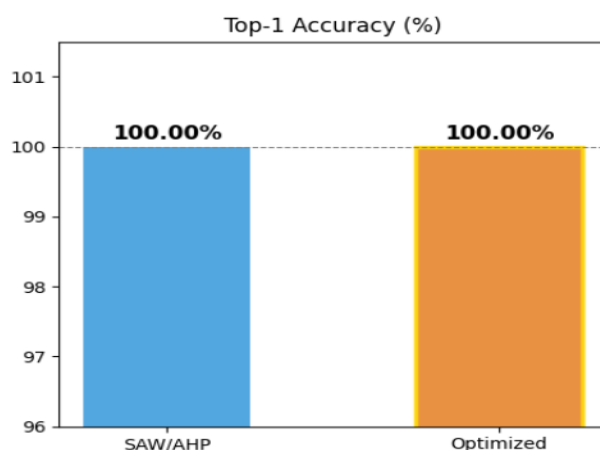
Hình 4.1 Trọng số sau 5 lần train

Qua 5 lần retrain với dữ liệu tích lũy dần, hai chiều semantic và attribute cho thấy xu hướng hội tụ rõ ràng: semantic giảm dần từ 0.4819 xuống 0.3741 trong khi attribute tăng từ 0.2411 lên 0.3348. Điều này phản ánh đặc điểm dữ liệu theo thời gian – khi mutation ngày càng đa dạng hơn, tín hiệu từ text hiển thị đơn thuần trở nên ít đủ để phân biệt candidate, buộc model học thêm trọng số từ thuộc tính HTML. Visual và Structure dao động nhẹ quanh mức ổn định. Context luôn dưới 0.02, cho thấy ngữ cảnh DOM lân cận ít

mang thông tin phân biệt trên dataset hiện tại. Sự hội tụ này cho thấy hệ thống online learning hoạt động đúng kỳ vọng: trọng số tự điều chỉnh phản ánh phân phối thực tế của dữ liệu healing.

4.7 So sánh hệ thống sử dụng trọng số tĩnh (SAW/AHP) và hệ thống khi sử dụng trọng số được tối ưu bằng mô hình học máy

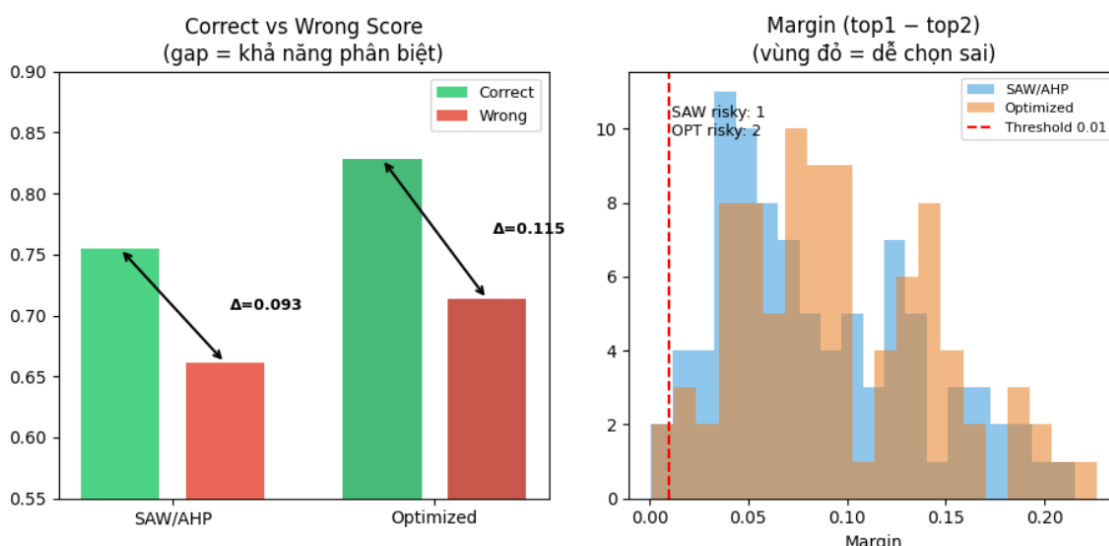
Phân tích so sánh được thực hiện trên **98 healing event từ event #101 trở đi** (362 candidates) — phần inference thực sự sau khi model đã được train.



Hình 4.2 So sánh Top-1 Accuracy

Top-1 Accuracy. Cả hai đều đạt **100%** — không có event nào chọn sai candidate trong 98 event post-training. Nhưng câu hỏi là: Cái nào tự tin hơn?

So sánh score gap (Khoảng cách giữa ứng viên đúng và ứng viên sai

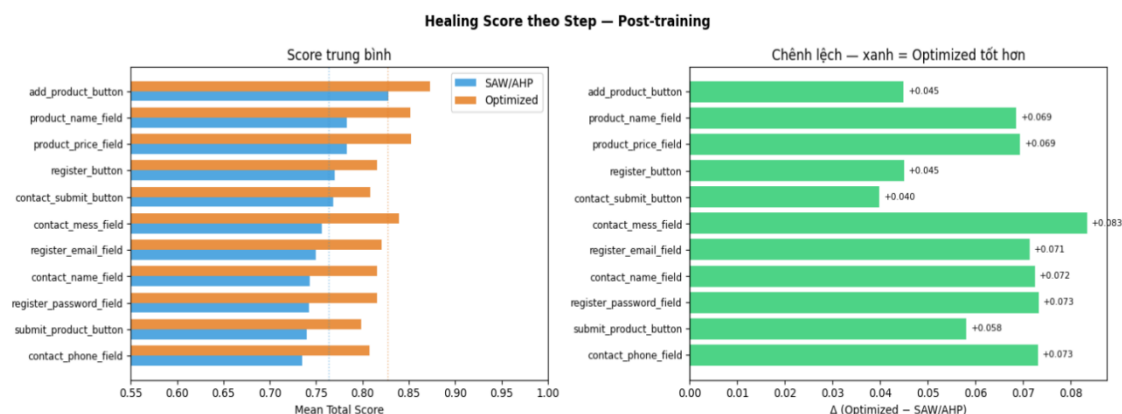


Hình 4.3 So sánh score gap trung bình

Score gap của Optimized vẫn cao hơn rõ ràng — khoảng cách giữa candidate đúng và sai lớn hơn SAW, thể hiện khả năng phân biệt tốt hơn. Margin distribution: OPT có 2

event risky (<0.01), SAW có 1 event risky, nhưng OPT có phân phối dịch phải hơn — tức nhiều event có margin an toàn hơn.

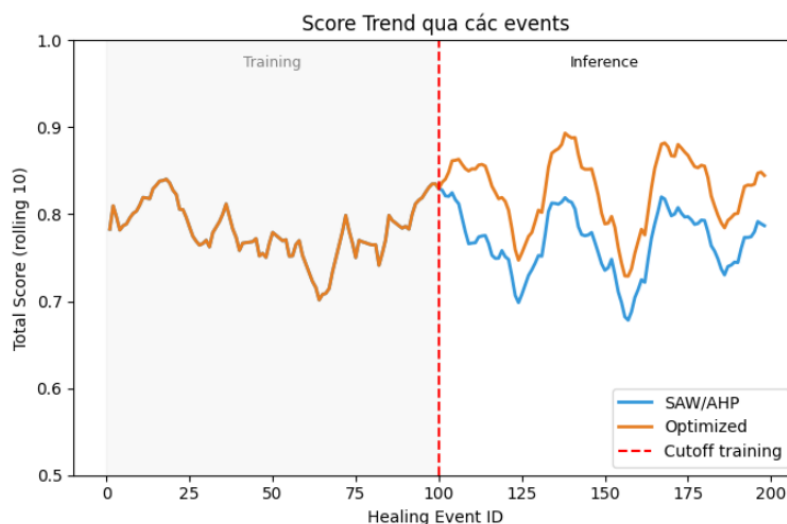
Score theo từng step



Hình 4.4 Score theo từng step

Model(Optimized) tốt hơn trên tất cả 11 steps, không có regression nào. Cải thiện lớn nhất ở `contact\_mess\_field` (+8.3%) và `register\_password\_field` (+7.3%) — những step có nhiều candidate text tương tự nhau mà model học được cách phân biệt tốt hơn sau training.

So sánh trước và sau training:

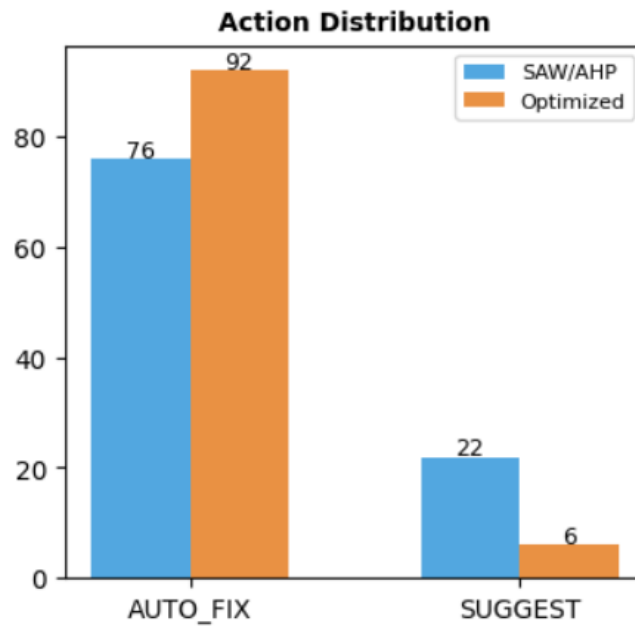


Hình 4.5 Score trước và sau train

Trước training, SAW và Model (Optimized) có score bằng nhau (0.7711) — vì chưa có gì khác biệt. Sau training, Optimized vọt lên 0.8281 trong khi SAW giảm nhẹ xuống 0.7546 — cho thấy SAW không học được gì thêm, còn Optimized cải thiện rõ rệt.

Đường rolling score cho thấy Optimized vượt qua SAW ngay sau mốc 100 và duy trì khoảng cách ổn định đó cho đến event cuối.

So sánh phân phối Auto_fix và Suggest



Hình 4.6 So sánh Auto_fix và suggest

Optimized tự tin hơn rõ rệt, AUTO_FIX tăng từ 76 lên 92 trong khi SUGGEST giảm từ 22 xuống còn 6.

Kết luận: Nhìn tổng thể, Optimized thắng trên tất cả các chỉ số sau training - không phải vì feature tốt hơn mà vì weighting tổng hợp tốt hơn và học được cách kết hợp các component hiệu quả hơn sau 100 events training.

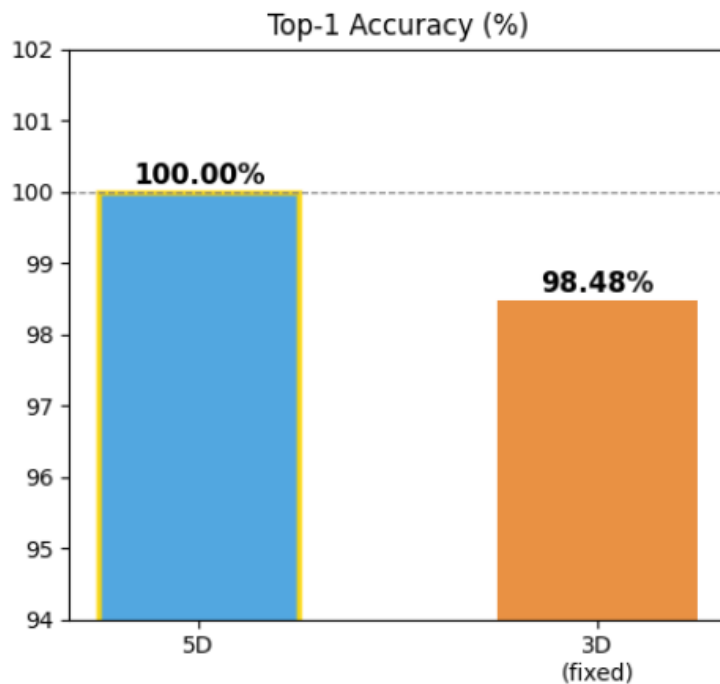
⇒ Nên sử dụng Model tối ưu cho production

4.8 So sánh mô hình 5 chiều và mô hình 3 chiều

Phân tích so sánh được thực hiện trên 198 healing event từ event (750 candidates)

- Mô hình 5 chiều sử dụng công thức $\text{attr} \times 0.27 + \text{sem} \times 0.24 + \text{struct} \times 0.20 + \text{visual} \times 0.12 + \text{ctx} \times 0.17$
- Mô hình 3 chiều bỏ hoàn toàn hai dimension cuối và redistribute trọng số: $\text{attr} \times 0.39 + \text{sem} \times 0.33 + \text{struct} \times 0.28$.

So sánh Top-1 Accuracy



Hình 4.7 So sánh Accuracy 5 chiều và 3 chiều

Nhận xét: 5 chiều đúng trên toàn bộ tập dữ liệu 198 heal event, 3 chiều chỉ đúng có 98,48% (3 heal event sai) yếu hơn đáng kể so với 5 chiều.

Những heal event bị heal sai ở mô hình 3 chiều:



Hình 4.8 So sánh những heal event bị heal sai ở mô hình 3 chiều với 5 chiều

- Event #63 — khoảng cách giữa "TẤT CẢ" và "Đăng xuất" ở 5 chiều là khá lớn (0.7290 vs 0.6629, $\Delta=0.066$), trong khi ở 3 chiều hai ứng viên này gần

như ngang bằng nhau (0.7428 vs 0.7410, $\Delta=0.0018$) — một khoảng cách nhỏ đến mức chỉ cần một biến động nhỏ cũng có thể đổi kết quả.

- Event #118 — đây là case nghiêm trọng nhất về mặt độ lệch. 5 chiều xếp "Gửi yêu cầu hỗ trợ" lên #1 với điểm 0.6944, nhưng 3 chiều lại đưa "Đăng xuất" lên 0.7498 — cao hơn cả điểm mà 5 chiều trao cho candidate đúng. Không phải 3 chiều "không chắc", mà 3 chiều *rất tự tin* — nhưng tự tin sai.
- Event #182 — tương tự, 3 chiều cho "Đăng xuất" score 0.7132 trong khi "ĐẢNG SẢN PHẨM LÊN HỆ THỐNG" chỉ được 0.6975. Điểm của candidate đúng ở 3 chiều thậm chí còn cao hơn điểm của nó ở 5 chiều (0.6466), nhưng vẫn thua vì "Đăng xuất" được inflate nhiều hơn.

Điểm mấu chốt để nhấn mạnh: Cả 3 event đều có chung một pattern — "Đăng xuất" có data-testid ổn định qua mọi UI version nên attr_score luôn cao. Khi attr chiếm 39% (thay vì 27%), nó áp đảo các dimension ngữ nghĩa còn lại. ctx_score là thứ duy nhất giúp 5 chiều nhận ra rằng "Đăng xuất" không phù hợp với *ngữ cảnh hiện tại* của step, còn visual_score giúp phân biệt vị trí layout. Bỏ cả hai đồng nghĩa với việc model chỉ còn "nhìn thấy tên và thuộc tính" của element, mà không còn hiểu element đó đang xuất hiện ở *đâu và trong hoàn cảnh nào*.

Kết luận: Sử dụng mô hình 5 chiều để có cái nhìn tổng quan và tối ưu hơn cho hệ thống.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Khóa luận đã nghiên cứu, thiết kế và triển khai hoàn chỉnh hệ thống Self-Healing UI Testing – hệ thống tự phục hồi kiểm thử UI web khi locator bị hỏng do thay đổi UI – tích hợp trực tiếp với Selenium WebDriver Python. Hệ thống giải quyết một bài toán thực tiễn quan trọng trong quy trình CI/CD hiện đại: giảm chi phí bảo trì test automation khi giao diện liên tục thay đổi mà không yêu cầu kỹ sư viết lại test case thủ công.

Các kết quả chính đạt được:

1. Xây dựng thành công hệ thống 5 module hoàn chỉnh (Exception Interceptor, Snapshot Store, Similarity Engine V2, Candidate Ranker, Knowledge Base) tích hợp liền mạch với Selenium WebDriver Python.
2. Đề xuất và triển khai mô hình tương đồng 5 chiều toàn diện (Attribute, Semantic, Structure, Visual, Context) với công thức tính điểm chi tiết cho từng chiều với công thức SAW/AHP, sau 100 heal event, tối ưu trọng số bằng mô hình học máy Logistic Regression (pairwise), đạt Top-1 Success Rate 100% trên bộ dữ liệu thực nghiệm.
3. Trọng số học bởi Logistic Regression tối ưu hơn SAW/AHP: Thực nghiệm so sánh có kiểm soát cho thấy trọng số ML (pairwise Logistic Regression, L2 regularization, train sau 100 healing events) cải thiện đáng kể so với trọng số SAW/AHP tĩnh: score gap tăng ~14%, AUTO_FIX tăng từ 76 lên 92 trên tổng 98 event, mean total score tăng từ 0.77 lên 0.828. Cả hai cùng đạt accuracy 100%, nhưng ML weights tự tin hơn và ổn định hơn.
4. Visual và Context là các chiều cần thiết để hệ thống có cái nhìn tổng quan hơn: Thực nghiệm so sánh mô hình 5 chiều với phiên bản rút gọn 3 chiều (bỏ Visual + Context) xác nhận sự cần thiết của hai chiều này. Model 3 chiều bị 3 lỗi nghiêm trọng – chọn nhầm nút "Đăng xuất" ở 3 healing event khác nhau do attr_score inflate lên 0.39 (từ 0.27) và không có ctx_score để counterbalance. Visual và Context đóng vai trò *tiebreaker ngữ nghĩa* – phân biệt element trong cùng DOM dựa trên vị trí hiển thị và ngữ cảnh xuất hiện

5.2 Hạn chế của nghiên cứu

Bên cạnh các kết quả đạt được, nghiên cứu còn một số hạn chế cần được thừa nhận:

1. Quy mô dữ liệu: Bộ dữ liệu 198 events từ 1 ứng dụng demo tương đối nhỏ. Tính khái quát hóa sang ứng dụng phức tạp hơn (nhiều trang, nhiều component) chưa được kiểm chứng.

2. Đa dạng mutation: Nghiên cứu tập trung vào các mutation có thể mô hình hóa bằng quy tắc. Các thay đổi UI phức tạp trong thực tế (thay đổi layout hoàn toàn, thêm màn hình mới) chưa được xử lý.
3. Multicollinearity giữa các sub-score: Các chiều Attribute, Semantic và Structure có tương quan cao nhau trên cùng một tập dữ liệu, khiến hệ số Logistic Regression không ổn định khi phân phối dữ liệu thay đổi. Regularization L2 hiện tại ($C=1.0$) là thiết lập mặc định, chưa được tối ưu qua cross-validation.
4. Phụ thuộc vào bounding box và trạng thái hiển thị: Visual score yêu cầu phần tử phải hiển thị và có bounding box hợp lệ ($\text{bounding_w} > 0$). Các phần tử ẩn (`display:none`), off-screen hoặc lazy-loaded sẽ nhận điểm neutral 0.20 — không phản ánh đúng mức độ tương đồng thực sự. Trường hợp test chạy trong môi trường headless với viewport khác nhau cũng ảnh hưởng đến tính nhất quán của tọa độ tương đối.
5. Semantic score dựa trên SequenceMatcher: Chiều Semantic hiện dùng SequenceMatcher (character n-gram overlap) để so sánh text và nearby_label. Phương pháp này không hiểu đồng nghĩa — "Gửi" và "Submit" sẽ cho điểm thấp dù cùng nghĩa. Đây là điểm yếu rõ ràng khi ứng dụng hỗ trợ đa ngôn ngữ hoặc có nhiều cách viết khác nhau cho cùng một action.

5.3 Hướng phát triển trong tương lai

Dựa trên kết quả và hạn chế hiện tại, khóa luận đề xuất các hướng phát triển tiếp theo:

5.3.1 Cải thiện mô hình tương đồng

- Tích hợp embedding ngôn ngữ tự nhiên (Sentence-BERT, OpenAI embeddings) cho chiều Semantic thay vì chỉ SequenceMatcher, giúp nhận diện các phần tử có cùng nghĩa nhưng khác từ ngữ.
- Chiều Visual có thể được nâng cấp từ việc so sánh bounding box thuần túy sang **so sánh screenshot cấp phần tử** bằng SSIM (Structural Similarity Index) hoặc perceptual hashing. Cách tiếp cận này cho phép phát hiện các phần tử có vị trí và kích thước tương tự nhưng hình thức hiển thị khác nhau do thay đổi CSS theme.
- Xây dựng chiều DOM Path Similarity dựa trên tree edit distance để đo sự tương đồng cấu trúc cây sâu hơn.

5.3.2 Cải thiện mô hình học máy

- Regularization mạnh hơn (ElasticNet) để giải quyết multicollinearity và tạo trọng số ổn định hơn.

- Ngoài ra, có thể thêm bước cross-validation để tự động chọn siêu tham số C phù hợp với từng ứng dụng cụ thể.

5.3.3 Mở rộng phạm vi

- Mở rộng sang framework kiểm thử khác: Playwright, Cypress, Appium (mobile testing).
- Thử nghiệm trên các ứng dụng thương mại thực tế với quy mô lớn hơn (>1000 test cases).
- Xây dựng plugin cho IDE (VS Code, IntelliJ) để kỹ sư có thể xem lý do healing và chấp nhận/từ chối đề xuất ngay trong môi trường phát triển.

5.3.4 Tối ưu hiệu năng

- Tăng tốc bằng vectorization: tính toán similarity hàng loạt cho nhiều candidate song song bằng numpy.
- Cache intermediate results: lưu cache bounding box và thuộc tính DOM để tránh query Selenium nhiều lần.

TÀI LIỆU THAM KHẢO

- [1] M. Leotta, A. Stocco, F. Ricca, and P. Tonella, "ROBULA+: An algorithm for generating robust XPath locators for web testing," *Journal of Software: Evolution and Process*, vol. 28, no. 3, pp. 177–204, 2016.
- [2] A Stocco, M. Leotta, F. Ricca, and P. Tonella, "PESTO: Automated migration of DOM-based Web tests towards the visual approach," *Software Testing, Verification and Reliability*, vol. 28, no. 4, e1665, 2018.
- [3] S. Choudhary, M. Versee, and A. Orso, "WATER: Web application test repair," in *Proc. First International Workshop on End-to-End Test Script Engineering (ETSE)*, pp. 24–29, 2011.
- [4] M. Hammoudi, G. Rothermel, and A. Stocco, "WATERFALL: An incremental approach for repairing record-replay tests of web applications," in *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2016)*, Seattle, WA, USA, 2016, pp. 751–762.
- [5] C. Burges *et al.*, "Learning to rank using gradient descent," in *Proceedings of the 22nd international conference on Machine learning*, 2005, pp. 89–96.
- [6] T. T. Nguyen, F. Ricca, and A. Stocco, "Automated web element locator evolution using simple search techniques," in *Proc. International Conference on Software Quality, Reliability and Security (QRS)*, pp. 20–30, 2021.
- [7] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*. New York: McGraw-Hill, 1980.
- [8] C. C. Aggarwal, *Machine Learning for Text*. Cham: Springer International Publishing, 2018.
- [9] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [10] W3C Web Accessibility Initiative, "Accessible Rich Internet Applications (WAI-ARIA) 1.1," W3C Recommendation, 2017. [Online]. Available: <https://www.w3.org/TR/wai-aria-1.1/>
- [11] SeleniumHQ, "Selenium WebDriver Documentation," 2024. [Online]. Available: <https://www.selenium.dev/documentation/webdriver/>
- [12] Y. Gong *et al.*, "Vision-based framework for automated web test generation and self-healing," in *Proc. International Conference on Software Maintenance and Evolution (ICSME)*, pp. 110–121, 2020.

- [13] A. Prasad and K. Muppala, "Healenium: Self-healing library for Selenium-based automated tests," GitHub repository, 2021. [Online]. Available: <https://github.com/healenium/healenium-web>
- [14] L. Yandrapally, S. Thummalapenta, S. Sinha, and S. Chandra, "Robust test automation using contextual clues," in Proc. 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), pp. 304–314, 2014.
- [15] Fishburn (1967): P. C. Fishburn, "Additive utilities with incomplete product sets: Application to priorities and assignments," *Operations Research*, vol. 15, no. 3, pp. 537–542, 1967.